HTL Leonding
next level

HÖHERE TECHNISCHE
BUNDESLEHRANSTALT
LEONDING

HitScan - Dart System

DIPLOMA THESIS

submitted for the

Reife- und Diplomprüfung

at the

Department of IT-Medientechnik

Submitted by:
Marvin Anderson
Luca Zinhobel

Supervisor:
Dietmar Steiner

Leonding, April 7, 2026

I hereby declare that I have composed the presented paper independently and on my own, without any other resources than the ones indicated. All thoughts taken directly or indirectly from external sources are properly denoted as such.

This paper has neither been previously submitted to another authority nor has it been published yet.

The presented paper is identical to the electronically transmitted document.

Leonding, April 6, 2026 Marvin Anderson & Luca Zinhobel

Abstract

In this thesis, a low-cost automatic scoring system for steel-tip darts is introduced. This system utilizes a single consumer camera and a web-based user interface. The system aims to retain the traditional experience of playing darts while maintaining automatic scoring. The system utilizes classical computer vision techniques based on OpenCV. The system processes video feed to locate the dart tip's coordinates. This information is then mapped to the dartboard's different regions. Both manual and marker-based approaches are considered to ensure ease of use and geometric accuracy. The system's software design utilizes a Python-based detection system, WebSocket event streaming, and a SvelteKit-based web application with storage.

The system's performance was tested in a controlled environment. The system was accurate in 70% of 103 throws at sector level. The system was accurate in 78.2% in the primary configuration and 60.4% in the low-resolution configuration. The system's performance was affected by the low resolution. This shows that the system is suitable in a home setting when factors such as camera stability and lighting conditions are well controlled. The system's limitations include accuracy at the boundary regions, high vibration conditions, and bounce-out cases. The system's performance shows that it is possible to design a deterministic single-camera system. This system can operate in real time without the use of specialized hardware.

Contents

1. Introduction

1.1. Motivation

The sport of darts is a combination of accuracy, concentration, and interaction with people. This sport has millions of players across the globe, both at professional and amateur levels. The digital revolution has affected many sectors within the arena of recreational sports to make them more efficient through data collection. However, the sport of darts at the amateur level remains at an analog level in terms of the process.

1.1.1. Lack of Affordable Automatic Systems

Apart from soft tip dart systems that use electronic boards, there is no broadly accessible automatic scoring solution for the traditional steel tip dartboard. Electronic soft tip systems change the physical playing experience and are therefore not an acceptable substitute for players who want to retain a classical bristle board. Existing automatic scoring products show that digital assistance is possible, but they remain outside the practical reach of many amateur players. This creates a clear gap between manual home play and professional-grade automated systems.

1.1.2. Limitations of Manual Score Entry

The most common alternative in steel tip darts is manual score entry, whether on paper, a whiteboard, or a calculator application. In all cases, players must repeatedly interrupt play to update scores and verify remaining checkouts. This breaks concentration, slows the rhythm of turns, and increases the likelihood of arithmetic mistakes under time pressure. In addition, manual workflows provide little structured match data, which limits feedback on consistency, checkout performance, and long-term progress.

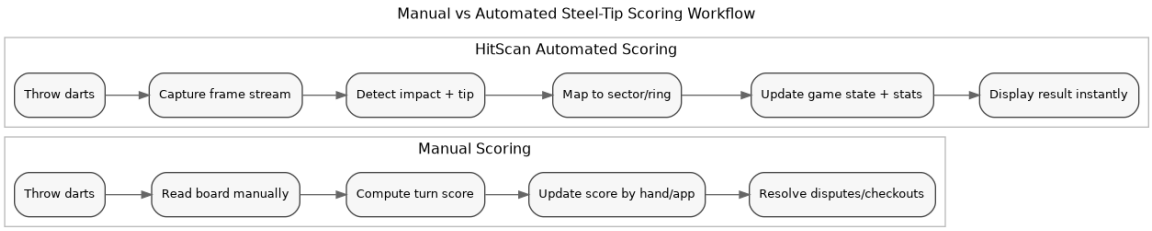


Figure 1.: Comparison between manual score tracking and the automated HitScan workflow.

From a user experience point of view, the difference is not simply one of convenience. Manual scoring requires repeated context switches between physical action and mental arithmetic. In a practice competition, repeated context switches can disrupt rhythm and negatively impact the transferability of practice to a situation that is more similar to a tournament environment. An automated pipeline ensures a smoother experience by keeping the player engaged in the motor task while providing accurate score progression and a full event history.

1.2. Problem Statement

This thesis is aimed at developing an automatic scoring system for the traditional steel tip dartboard. The system is designed to ensure that the physical nature of the dartboard is linked to the digital nature of the computer system while at the same time ensuring that the system is simple in terms of the hardware used. The entire system is designed to work with just one camera. Such a system is highly suitable for use in the home environment as it does not require any specialized equipment.

Real-Time Detection. In terms of practical usability, it should be able to detect these hits and display the updated score with minimal delay. Excessive delay would break the natural flow of the game and hinder usability. Thus, the delay between the dart landing and the display of the updated score should be well below a few seconds.

This constraint will affect the algorithm and hardware used. The image processing algorithm should be efficient and analyze the video stream to detect the dart landing. Once the landing has been detected, it should be able to identify which dart landed and calculate the resulting score with minimal delay to be considered instantaneous to the player.

1.3. Project Scope

1.3.1. Low-Cost Design

The first constraint of the system is its cost. The solution should remain substantially cheaper than existing automatic scoring systems and therefore rely on off-the-shelf consumer hardware. It should run on readily available devices such as a Raspberry Pi or a standard home computer without requiring dedicated accelerators or specialized sensing equipment.

1.3.2. Single-Camera Setup

To further limit cost and setup complexity, the system is restricted to a single-camera configuration. This scope decision imposes visibility constraints on the solution: the dartboard must remain fully visible, and the camera position must minimize occlusion at the moment of impact. The thesis therefore focuses on what can be inferred reliably from one two-dimensional video stream.

1.3.3. Scope Boundaries

In order to clarify the ambiguity of expectations, the scope of the project was clearly defined, including the features to be included and the features to be excluded.

Table 1.: In-scope and out-of-scope boundaries for this thesis.

In Scope	**Out of Scope**
Single-camera steel-tip detection on consumer hardware	Multi-camera 3D triangulation and stereo reconstruction
Local-network web interface and real-time score display	Cloud-based user accounts, remote matchmaking, social platform features
Calibration-assisted sector mapping (manual and marker-based)	Automatic bounce-out recognition with guaranteed reliability
Core game modes (301/501 and common checkout rules)	Full support for all casual and tournament game variants

The definition of the scope of the project is critical to the interpretation of the results of the evaluation process. Certain failure modes, such as bounce-outs and complex

occlusions, are considered to be a limitation of the architecture, not a regression against the specified requirements.

1.4. Objectives

1.4.1. Functional Objectives

Automatic Hit Detection

The main goal is to create a software module that can detect the moment a dart hits the target and the section it has hit. It analyzes the video stream, finds a new stationary dart, and maps it to the correct section.

Interactive Web Interface

It is necessary to implement a web interface that can be accessed on the same computer that analyzes the video stream. Players can interact with it using other devices connected to the same local network, such as smartphones or tablets.

Support for Multiple Game Modes

The developed application supports different versions of the game. Most commonly used versions include 501 and 301. In addition, the following rule variations have been implemented: Double In and Double Out. These versions cover the most popular ones, making it easier to play with the developed software.

Transparent Correction and Recovery Workflow

Since it is impossible for the auto hit detection mechanism to be 100% effective under all circumstances, a correction workflow that is readily available and easy to understand for the user is a necessity. This means that a workflow for undoing actions is a required part of the objectives, not a secondary addition. This allows players to easily recover from a rare false positive without interrupting the flow of the game.

Persistent Player-Centric Statistics

In addition to displaying the scores, the software will also need to include a player-centric statistics system that can be understood by the average user. This means converting

the raw data of throws into useful information, such as hit percentage, win percentage, distribution, etc., so that the software can be useful for gameplay as well as performance analysis of a player's throws.

1.4.2. Technical Objectives

Motion-Based Dart Recognition

The software will need to recognize when a dart hits the board, differentiate it from the static scene, and accurately recognize where it landed for automated scoring.

Manual and Marker-Based Calibration Concepts

Since the software will need a calibration step where it maps the image coordinates onto the dartboard's scoring areas, a manual workflow and a marker workflow will be considered. This will allow the team to weigh the trade-offs between simplicity and precision when it comes to the calibration process.

Deterministic Frontend State Transitions

Since the frontend will need to apply the rules of the game deterministically, meaning that it will need a set of rules for when a turn changes, when a player busts, and when a player wins, determinism is a technical objective.

1.4.3. Objective Traceability

To ensure methodological consistency, each of the high-level objectives was carefully matched with the technical mechanisms supporting it and then validated in the evaluation chapter.

Table 2.: Traceability from objectives to implementation mechanisms and evaluation evidence.

Objective	Implementation Mechanism	Evaluation Evidence
Automatic hit detection	Motion threshold pipeline with contour-based tip estimation	Sector-level accuracy and failure analysis in evaluation chapter
Real-time score feedback	Backend event emission and frontend state update through WebSocket	Practical responsiveness observations during play sessions
Reliable board mapping	Manual/marker calibration and coordinate transformation	Calibration-related error categories and boundary behavior analysis
Low-cost accessibility	Single-camera, CPU-only classical CV pipeline	Feasibility on consumer hardware without dedicated accelerators

1.4.4. Acceptance Criteria Snapshot

The following minimum acceptance criteria were used as a practical checklist during the iterative development cycles.

Listing 1: Practical acceptance checklist used during iterative testing

```
1 Full board remains visible in camera frame under chosen mount.
2 Calibration can be completed and persisted without manual file edits.
3 New dart impact triggers a valid WebSocket hit event.
4 Detected hit maps to a ring-sector result consumable by game logic.
5 Scoreboard updates without user-side refresh or manual score input.
6 Pipeline remains usable under stable indoor lighting and typical throws.
```

1.5. Methodology

1.5.1. Prototype-Based Experimentation

The project will adopt an iterative development approach. First, the minimal viable prototype for the vision subsystem will be developed. This will help in validating the initial assumptions about the perspective of the camera and the detectability of the darts. Secondly, the prototype for the web interface will also be developed.

1.5.2. Evaluation and Refinement Cycles

The project will undergo various refinement cycles. Each subsystem, such as the vision, game logic, and interface, will be individually implemented and tested against the requirements. The feedback from the testing process will directly influence the subsequent phases of the project, resulting in the creation of a robust, user-friendly interface, as discussed in the project timeline.

1.5.3. Methodological Workflow

The development process used an iterative measure and adjust cycle instead of a linear one-shot approach to development. In each cycle, a specific major uncertainty was identified (for instance, thresholds' stability, calibrations' repeatability, and camera angles' sensitivity), isolated through specific tests, and incorporated only after verifying its acceptable behavior.

Listing 2: Iteration loop applied throughout development

```
Cycle Input: one unstable system behavior
  Define the expected behavior and a corresponding measurable metric.
  Instrument and debug the corresponding stage (for instance, saved masks,
      contours, and tip output).
  Modify one parameter group at a time.
  Re-run specific controlled throw tests (center, edge, out-of-bounds).
  Log the behavior and decide to commit or revert the modification.
  Integrate only when no regression is seen on previously stable behavior.
```

This approach reduced the risk of "regressions" where improvements to one component of the system's detection pipeline might have a negative impact on another component when tested under different lighting, angle, and/or calibration conditions.

2. Related Work and Market Analysis

2.1. Commercial Dart Scoring Systems

2.1.1. High-End Multi-Camera Systems

Technical Capabilities

The technical capabilities of commercial automatic scoring systems used in televised darts tournaments are based on a multi-camera triangulation methodology. A minimum of three high-speed cameras is used to track the trajectory of the dart. This methodology is in line with prevalent methodology in sports tracking research, where the reconstruction of fast-moving objects in three-dimensional space is necessary [1]. These cameras are operated at high frame rates to eliminate motion blur and accurately track the trajectory of the dart.

In order to eliminate occlusions, the trajectory of the dart is tracked using model-based tracking with Kalman filter estimation during periods of invisibility [2]. The region of impact on the dartboard is accurately determined using subpixel resolution.

Cost and Setup Requirements

The technical capabilities of the system require setting up a multi-camera system with camera calibration. Camera calibration is important in order to ensure accurate results from the system. If the cameras are not accurately aligned, the system will not produce accurate results [3].

As a result, the system is complex and expensive, making it unsuitable for home use. This complexity in setting up the system is one of the factors that prompted the consideration of alternative methodologies based on a single-camera system.

Table 3.: Comparative overview of dominant automatic scoring approach families.

Approach	Primary Strength	Primary Weakness	Consumer Suitability
Multi-camera vision systems	High spatial observability and strong occlusion robustness	High setup/calibration complexity and high hardware cost	Low
Sensor or laser-based systems	Fast event triggering and reduced image-processing load	Material/mount dependency and limited visual verification	Medium
Single-camera classical CV (this work)	Low cost, maintainable stack, no training data dependency	Boundary sensitivity and bounce-out limitations	High

2.1.2. Laser and Sensor-Based Systems

Hardware Dependencies

The alternative systems that have been made available in the market use laser technology or a sensor that is attached to the dartboard. In this system, the position of the dart is determined by the interruption of laser beams or the propagation of vibrations. A similar technology is used in the impact localization system, in which several sensors are used to detect the position of the dart based on the time difference in the impact detection process [4].

Accuracy Characteristics

The sensor-based system provides reliable results for steel darts but also depends upon material properties and damping effects. In this system, the wave propagation also depends upon the wear and tear of the dartboard, resulting in localization error. In addition, this system cannot detect bounce-outs or mid-air collisions. Moreover, it also fails to provide visual verification.

2.2. Research on Sports Object Tracking

2.2.1. Ball and Projectile Tracking

Motion Segmentation Techniques

Motion segmentation techniques have been widely used in video analysis to segment moving objects in a given video. Background subtraction techniques, including adaptive Gaussian mixture model-based techniques, have been used to segment moving objects from the background [5].

These techniques have been widely used in sports object tracking to detect moving objects such as balls or projectiles. Edge detection techniques have also been used to improve the accuracy of localization. In edge detection techniques, object contour detection is used to improve localization accuracy. In object contour detection, edges in the image or video frame are detected to improve localization accuracy [6]. For circular objects, a Hough transform-based detection technique can also be used [7]. These techniques have been used to detect moving objects in a video frame.

Trajectory Estimation Methods

After detecting a moving object in the image or video frame, its trajectory can be estimated using state estimation techniques. In state estimation techniques, a Kalman filter-based technique can be used to estimate the trajectory of the object in the image or video frame [2].

This method is advantageous for detecting temporarily occluded balls or for estimating the position of ball impact within a sports environment [1]. The integration of object detection and trajectory estimation increases the capability for detecting a high-contrast moving object within a video stream.

2.3. Existing Open Source Projects

2.3.1. Dartboard Detection Tools

Marker-Based Alignment

Fiducial markers have been used for camera calibration in various research prototypes. ArUco markers provide accurate pose estimation, especially in the presence of partial

occlusion. By using multiple markers on the dartboard, it is possible to compute the homography between image coordinates and dartboard coordinates. This enables the computation of impact point coordinates in standard dartboard coordinates, irrespective of the camera perspective.

Geometric Modeling Approaches

Most systems, after camera calibration, have used geometric modeling for dartboard alignment. The geometric model consists of concentric rings and radial sectors. By identifying the nearest ring and sector boundaries, it is possible to determine the hit location. This approach is advantageous over machine learning-based methods, as it uses a deterministic process instead of mapping pixel coordinates to dartboard regions, making it more suitable for real-time applications due to lower computational costs.

2.4. Position of This Work in Current Literature

2.4.1. Comparison with Existing Approaches

Hardware Differences

From the existing approaches, it is evident that using multiple hardware configurations for camera placement and using sensor-based systems instead of image processing provide trade-offs for robustness. The presence of multiple cameras increases the spatial observability, while the sensor-based system avoids image processing altogether but is dependent on the material and sensor placement conditions.

Novelty of a Motion-Threshold Pipeline

In this space, classical computer vision offers a relevant alternative to learning-based detection. Past work on motion segmentation, contour-based localization, and geometric board modeling has shown that a deterministic process can remain effective in constrained conditions. For this reason, the technological justification for this direction is not repeated here.

2.4.2. Comparative Decision Criteria

Beyond pure detection accuracy, practical deployment requires balancing setup burden, interpretability, and maintenance overhead.

Table 4.: Qualitative decision matrix used to position the chosen approach.

Criterion	Multi Cam	Sensor/ Laser	ML Vision	Classical CV	Priority in This Thesis
Hardware cost	High	Medium	Medium	Low	Low
Installation simplicity	Low	Medium	Medium	High	High
Model/data dependency	Low	Low	High	Low	Low
Explainability	Medium	Medium	Low	High	High
Recreational-home fit	Low	Medium	Medium	High	High

The matrix highlights why a deterministic single-camera pipeline was selected. It does not maximize every individual metric, but it aligns best with the objective profile of low-cost recreational deployment under constrained hardware.

3. Foundations

3.1. Camera Models and Projective Geometry

3.1.1. Perspective Projection

Pinhole Camera Model

The camera image is a projection of the real-world scene onto the two-dimensional image plane. The projection is achieved according to the pinhole camera model, in which each point in the real world is projected to the image sensor through the camera center as a straight line [3].

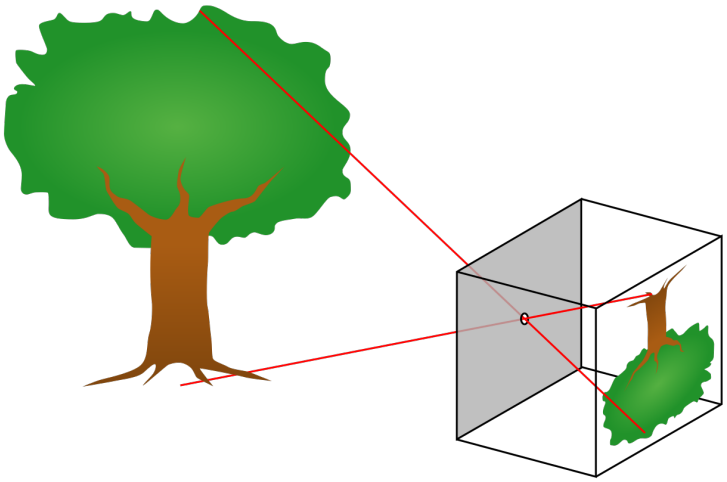


Figure 2.: Pinhole camera projection model. Public domain image from Wikimedia Commons, original author: DrBob, redraw: Pbroks13.

Due to the projection nature, the measurements in the image are not proportional to the real-world measurements. Far objects are perceived as smaller than near objects. Even circular objects may be perceived as elliptical depending on the viewpoint. Such is the situation with the dartboard, as it is planar but viewed from an angle [3].

Lens Distortion and Correction

The real lens does not follow the pinhole camera model. Due to lens distortion, lines in the real world are perceived as curved lines in the camera image. The distortion is

most pronounced at the image boundaries. If not corrected for, the dartboard would be mislocated in the image, causing the wrong dartboard section to be registered as hit. Thus, camera calibration is required to correct for lens distortion before any geometric processing is performed [8].

3.1.2. Homography and Planar Mapping

Coordinate Transformations

The dartboard is planar, so the projective transformation is used to map the image coordinates to the dartboard coordinates [3].

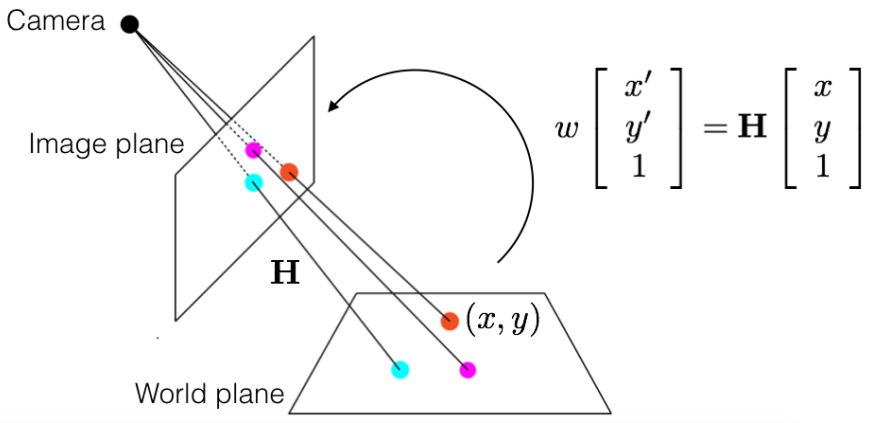


Figure 3.: Planar homography mapping between camera image and world plane. Image by Faisal Qureshi, Ontario Tech University, licensed under CC BY-NC 4.0.

Image-to-World Mapping

During calibration, reference points on the board are matched with their image positions. This correspondence defines a mapping from camera pixels to board coordinates, allowing later detections to be interpreted in the dartboard reference frame [3].

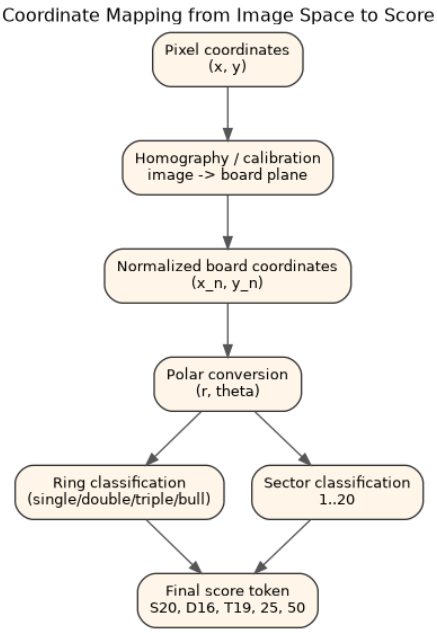


Figure 4.: Conceptual mapping chain from camera pixels to final score token.

3.2. Dartboard Geometry

3.2.1. Ring and Sector Structure

Standard Dartboard Dimensions

A dartboard has a series of scoring rings and sector divisions. The dimensions of the dartboard have a constant ratio with regard to each other. Practice boards can have a constant scale for all dimensions. This means that the detection process is independent of the size of the dartboard.

Sector Indexing and Score Mapping

Once a point has been expressed in board coordinates, its radius and angle determine the corresponding ring and numbered sector.

3.2.2. Coordinate Systems

Pixel Space

The camera provides a position in pixel space. This is a function of the camera's perspective. This information is not suitable for scoring.

Normalized Board Space

After calibration, detected points can be represented in a normalized board coordinate system that is independent of the raw image perspective.

Table 5.: Coordinate representations used across scoring stages.

Space	Typical Variables	Purpose
Pixel space	(x, y)	Raw sensor-domain localization output
Board-normalized planar space	(x_n, y_n)	Perspective-independent geometric reasoning
Polar board space	(r, θ)	Direct ring/sector decision mapping
Symbolic scoring space	`S20`, `D16`, `T19`, `25`, `50`	Game-rule compatible hit representation

3.3. Principles of Motion Detection and Object Tracking

3.3.1. Frame Differencing

Temporal Stability

Motion-based detection assumes a static camera and a stable scene. Otherwise, global image changes would be indistinguishable from the local motion caused by a dart impact.

Noise Sensitivity

Lighting conditions play an important role in motion detection. Reflections and shadows may be mistaken for movement. Therefore, stable lighting and a stable background are necessary for reliable detection.

3.3.2. Blob Detection and Tracking

Contour Extraction

After frame differencing isolates moving regions, contour extraction describes their outlines so that candidate dart impacts can be localized geometrically.

Filtering Strategies

Filtering suppresses noise and removes implausible regions before further localization is attempted.

3.3.3. Boundary Sensitivity Considerations

In a discretized scoring model, localization errors are most harmful near sector and ring boundaries. A small perturbation in estimated tip position can produce a categorical score jump, even if Euclidean displacement is small.

Listing 3: Boundary sensitivity intuition in discrete scoring systems

```
Given estimated tip p = (x, y) and decision regions R_i:

score(p) = i  if p in R_i

Near boundary B_ij between R_i and R_j:
small ||delta p|| can flip score(p + delta p) from i to j.

Implication:
Localization robustness must be evaluated categorically (correct region)
and not only by geometric distance error.
```

This property motivates both stable calibration and conservative filtering strategies in the implementation.

4. Technologies

4.1. System Requirements and Constraints

4.1.1. Real-Time Processing Requirements

Latency Constraints

The entire process from capture, through preprocessing, motion detection, blob extraction, tip detection, and scoring, needs to be completed within the time interval between two successive frames. Since the frame rate is 30 frames per second, the time constraint is 33 milliseconds, while at 15 frames per second, the time constraint is 66 milliseconds [9]. It was decided that the end-to-end latency from the time the dart hits until the time the score appears on the screen needs to be less than 100 milliseconds. This was determined through informal testing, where it was observed that such high delays are noticeable during the course of play. Non-critical processing such as the creation of overlays and match logging is done after the commitment of the detection result and is not subject to the above constraints.

Pipeline Responsiveness

The responsiveness of the pipeline is maintained by making the more expensive operations dependent on the results obtained from the less expensive operations. Capture of the frame and the differencing operation are performed unconditionally on every frame. Subpixel tip detection, perspective correction, and sector assignment are performed on every frame, provided the differencing operation indicates motion greater than the specified threshold.

4.1.2. Hardware Constraints

Single-Camera Assumption

The system assumes the availability of a single camera. Restricting the problem domain to a single-camera setup was a conscious decision. This eliminates synchronization

problems, keeps the calibration process manageable for the end user, and minimizes hardware costs to a single USB camera module. In practice, spatial reasoning is limited to a single image plane. A hand occluding a dart or a previously lodged dart cannot be resolved geometrically. However, this is acceptable for the problem domain because the dart tip is typically visible from a frontal overhead position at the time of impact.

Static Mount Requirements

This detection approach is predicated on a static camera position between the time of calibration and use. This is necessary for the background model, the stored ArUco pose, and the pixel-to-board homography. Any physical movement of the camera, even slight, invalidates the stored calibration and causes a scoring problem due to a systematic error. A startup routine is implemented to ensure the presence of a valid stored calibration. If this is missing or marked as invalid, the system prompts a recalibration routine.

4.2. Programming Language and Runtime Environment

4.2.1. Python as Core Development Language

Computer Vision Suitability

Python has been selected due to the maturity of the OpenCV bindings. Every function in the performance-critical path, i.e., encompassing the camera grab, color conversion, morphology operations, contour detection, ArUco detection, and `solvePnP` execution, is performed within OpenCV's compiled C++ backend. Python is used to orchestrate this process. Hence, only function call overhead is incurred in the Python interpreter. This is consistent with the cost model we would expect for this type of real-time processing pipeline.

Scientific Ecosystem

This is because the underlying data structure of NumPy arrays matches the internal memory structure of OpenCV, allowing for efficient passing of frames between libraries without copying data. Additionally, the scipy library offered the necessary geometric primitives for implementing the mapping using polar coordinates during the early stages of prototyping. Matplotlib also helped in the inspection of intermediate results during the development stage, such as difference frames, binary masks, and contour

maps, without needing a separate debugging tool. The fact that the above libraries are available as drop-in modules greatly reduced the development cycles during parameter tuning.

4.2.2. Runtime Characteristics

Interpreter-Based Execution

The interpreter imposes a certain overhead on the execution of the Python bytecode in a loop over the pixel data. However, this was not a performance constraint, as no stage of the pipeline needs to loop over the pixel data at the Python level. All pixel array operations are vectorized using NumPy or OpenCV [10]. There is one loop at the Python level, which iterates over a small list of filtered contour candidates. This has negligible performance impact, as it needs to process at most a few objects per frame.

Performance Considerations

The first optimization step for the project involved the implementation of region of interest cropping. This involved limiting the processing of the image to the region of interest, which contains the dartboard. This resulted in a reduction of between 60-70 percent of the total number of pixels processed compared to the whole frame [11]. The output buffers are also pre-filled, which are then passed as arguments for OpenCV's in-place processing, thus eliminating the need for heap allocation. In the event that the early exit condition is met, with no motion detected, the processing pipeline takes a short time, less than 10 milliseconds, thus giving ample time for the UI thread.

Table 6.: Approximate per-stage runtime budget interpretation for practical operation.

Pipeline Stage	Typical Cost Class	Optimization Used
Frame acquisition + grayscale conversion	Low	Direct OpenCV capture path
Differencing + thresholding + morphology	Medium	ROI restriction and in-place operations
Contour extraction + tip estimation	Medium	Early exit when no motion is present
Score mapping + event emission	Low	Lightweight geometric classification
Overlay/debug composition	Low to Medium	Performed after detection commitment

4.3. Computer Vision Framework

4.3.1. OpenCV

Image Capture and Preprocessing

The images are captured using the OpenCV function `cv2.VideoCapture`. After that, the images are converted to grayscale, which is necessary for the detection. However, sometimes it is also necessary to keep the color information, for example, to filter the specular highlights on the metallic ring. In this case, the image is converted to the HSV space. Lens distortion is compensated using the intrinsic parameters obtained during the chessboard calibration. This is necessary for the outer scoring rings, where otherwise the apparent position of the darts on the board would be off by several millimeters due to the barrel distortion.

Filtering and Thresholding

A Gaussian blur is applied before the background subtraction step to remove sensor noise which could cause unwanted contour formation. Otsu's method is then used to set the binary threshold after background subtraction, which eliminates the need for retuning the fixed threshold value in response to changing ambient light conditions [12]. Morphological opening is then used to remove isolated single-pixel noise, which is followed by a closing function to fill in small gaps in dart-sized objects before

contour extraction. As a last resort for dealing with environments featuring significant illumination gradients across the board, adaptive thresholding is also supported, though at the expense of a small increase in processing time.

4.3.2. Supporting Libraries

NumPy for Numerical Computation

All coordinate data computed during the OpenCV pipeline, including contour points, homography, and marker corners, is represented as NumPy arrays and remains this way throughout the entire process. The computation of pixel coordinates from board coordinates is accomplished through a matrix multiplication followed by an `arctan2` function, which is performed over the entire set of candidate points simultaneously. This maintains the high speed of the coordinate computation step even when multiple candidates are present.

Geometry Utility Libraries

A utility library has been developed for handling the conversion between polar coordinates and sector coordinates, including identifying which numbered segment and which type of ring (single, double, triple, or bull) a given board coordinate represents. It uses a lookup table of angular and radial thresholds to represent the standard dartboard layout and exposes a single function which accepts a coordinate in board space and returns the corresponding score value. This separation of code from the detection routine also allowed for the independent unit testing of the dartboard scoring rules without the requirement for image resources.

4.4. Calibration Approaches

4.4.1. Marker-Based Calibration

ArUco Marker Detection

Four ArUco markers are placed at known positions around the board frame [13]. The corner points are identified in the camera view through `cv2.aruco.detectMarkers` with the ArUco dictionary `DICT_4X4_50`. The markers are uniquely identified with different IDs. Thus, even in the event that one marker is occluded, the corner points

are identified unambiguously. The corner points are then passed to the next stage for estimation.

Pose Estimation

The corner points are then used to obtain the camera pose with respect to the board through `cv2.solvePnP`. A homography matrix is then obtained that maps any point in the camera view to the corresponding point in the front view [3]. The homography matrix is stored for the entire duration as the camera is not moving.

4.4.2. Manual Calibration

User-Defined Ring and Sector Placement

The user may choose not to use the ArUco markers for calibration. Instead, the user is prompted to click the center of the bullseye in the live view window. Then the user is prompted to click any point on the outer double ring. The two points are used to calculate the center and radius of the dartboard in pixels. All the radial thresholds are then scaled from the derived radius according to the proportions in the conventional dartboard. After the rings are set up, the user selects one of the known sector boundaries. The known sector boundary between segments 20 and 1 located at the top of the board was used as the standard to measure the angular offset. The 20 sector boundaries are then evenly distributed with 18-degree increments of the reference angle. This is necessary since the board's rotational orientation with regard to the camera cannot be determined beforehand and will change depending on how the board is mounted.

4.4.3. Comparison of Calibration Methods

Setup Complexity

ArUco calibration requires the user to place four markers and attach them to the board before first-time usage. Once the markers are attached, the process runs automatically every time the board starts without further user intervention. Manual calibration requires no pre-installation but requires the user to interact with the program several times each time the board starts or the camera is moved. For a permanent installation, ArUco would be much simpler to set up; for an ad hoc setting, manual calibration would be preferable to the requirement of attaching four markers to the board.

Robustness in Real Environments

Based on the experiments performed, it was found that the ArUco method of calibration was more reliable and consistent with regard to the assignment of each sector between sessions since it takes into consideration the homography to correct for perspective distortion. The manual method would gradually accumulate error depending on the accuracy of the clicks and would not compensate for perspective problems. Although under controlled conditions the accuracy of the ArUco and manual method of calculating the homography would be nearly equivalent with regard to the accuracy of the points scored by each method, under less than optimal camera angles with regard to the board, the homography method would clearly produce fewer errors with regard to the correct classification of adjacent segments along the board boundaries.

Table 7.: Operational comparison of calibration strategies in this project.

Criterion	**Manual Calibration**	**ArUco Calibration**
Initial setup effort	Low hardware, higher interactive effort	Marker placement required once
Repeatability across sessions	Moderate, click-dependent	High, marker pose-dependent
Perspective compensation quality	Limited in oblique views	Strong through homography estimation
Best fit scenario	Ad hoc temporary setups	Fixed or semi-permanent installations

4.5. Motion Detection Techniques

4.5.1. Frame Differencing

Background Selection

The background model is pre-initialized using a median value computed over a short frame buffer taken before play begins, during which time no dart is on the board. A median rather than a mean value is used to resist transient effects of passing shadows or exposure changes [14]. During play, the background model is not updated since the dart

should be visible as a persistent foreground object. Background updating is invoked only when the user indicates that the darts have been removed from the board.

Temporal Stability

A single-frame difference is noisy by nature. A dart landing in a single frame and staying there in subsequent frames would result in a high difference in the landing frame and almost zero differences in subsequent frames. To avoid false alarms due to fluorescent lighting flicker and board surface vibrations, the system requires the blob to be present in at least two consecutive difference frames before it is considered a hit.

4.5.2. Threshold-Based Segmentation

Binary Mask Generation

Otsu's thresholding is used to convert the absolute difference between the current frame and the background model to a binary mask. This is quite applicable in this case, considering that the difference image is mostly dark with a small bright blob corresponding to the dart. This is a binary image problem, and Otsu's method is applicable in this case. This mask is the only input to the contour extraction process.

Sensitivity to Noise

There are two sources of noise in this problem. One is the specular reflection from the metallic ring between the double and treble regions. The other is pixel noise at low levels. Sensor noise is taken care of by the Gaussian blur applied before the differencing process. Specular reflection is taken care of by the morphological opening process that removes objects with sizes below a specified minimum area. This is set empirically based on the false positives encountered on the development board.

4.5.3. Blob Detection

Contour Extraction

The function `cv2.findContours` is called with the binary mask and the flag `RETR_EXTERNAL` to retrieve only the outermost contours while ignoring any nested ones. This is sufficient for the dart tip projection because it is a compact area with no inner regions. The

returned value is a list of contour points that describe the outline of each region in the image.

Filtering of Irrelevant Regions

The regions are filtered by four checks before any of the tip coordinates are computed. The first check is that of area. Regions that are either below the empirically determined minimum or exceed the maximum possible cross-section area are eliminated. The second check involves the aspect ratio. Highly elongated regions are unlikely to correspond to a dart tip when viewed from above. The third check involves the shape solidity, which is given by the ratio of the contour area to the area of the convex hull. Irregular shapes resulting from reflections or cables have lower solidity values. The fourth check involves the spatial location of the region. Regions whose centroids are outside the known region of the board are eliminated. The number of regions that remain at this point per frame is usually either zero or one, occasionally two, before the tip localization.

Table 8.: Key detector parameters and operational role.

Parameter	**Role in Pipeline**	**Tradeoff Direction**
Motion threshold	Separates background variation from candidate foreground	Higher values reduce noise but risk missed weak impacts
Minimum blob area	Rejects tiny artifacts and sensor noise	Higher values improve precision but can remove valid small tips
Morphology kernels	Connect fragmented dart regions before contouring	Larger kernels improve continuity but can over-merge nearby regions
Still-time gate	Waits for stable post-impact frame before analysis	Longer gating reduces blur artifacts but increases response delay
Known-dart suppression radius	Avoids repeated detection of already lodged darts	Larger radius reduces duplicates but may hide nearby new impacts

Listing 4: Parameter tuning strategy used during detector stabilization

```
Start from conservative threshold and high min-blob area.
Verify no false triggers on static board for several seconds.
Lower threshold until weak impacts become visible in mask.
Re-tune morphology to reconnect fragmented dart regions.
Validate edge and out-of-bounds throws before locking values.
```

4.6. Frontend and Visualization Technologies

4.6.1. Frontend Framework Selection

SvelteKit Architecture

The choice of frontend framework was driven by the need for a framework with a short development cycle, close integration with reactive state management, and good runtime performance on distributed client platforms. SvelteKit was chosen for this project due to its ability to compile declarative component definitions into optimized JavaScript code, bypassing the runtime package size penalty of virtual DOM frameworks like React and Vue.

Key Features of the Framework

The framework's first-class support for file-system-based routing, server-side rendering, and progressive enhancement enabled a compact codebase with good cross-platform compatibility on Windows and Linux. This combination of features is a good match for the requirements of the dart detection system's single-page game-like workflow.

Another advantage of this framework choice was the excellent integration with the TypeScript programming language. The framework's toolchain provided good support for TypeScript's type system, allowing for robust typing of route parameters, store payloads, and websocket event structures. This minimized integration errors with the backend code. In the early stages of development, this minimized runtime errors caused by incorrect data contracts.

The framework's support for separating route-level loading logic and interactive component logic was another advantage. This feature was useful for this project since the setup pages have a different lifecycle than the game route. This separation of concerns at the route level was useful for correctness reasoning.

Comparison with Other Frameworks

Other frameworks, like Angular and Next.js, were considered for this project but rejected in favor of SvelteKit due to their larger runtime bundle and configuration complexity for a single-page game-like workflow. SvelteKit's lightweight nature made it a more suitable choice for this project.

Alternatives using React were not discounted on technical merit but do pose an extra layer of decision-making in terms of state libraries, routing strategies, and rendering paradigms. As a thesis project with limited code development time, reducing architectural decisions was a direct productivity gain. The SvelteKit framework provided sensible defaults to allow focus on application-specific problems: synchronization, accuracy in ruleset definitions, and analytics data quality.

4.6.2. Real-Time Communication Strategy

WebSocket Transport Selection

The real-time communication strategy was to utilize the WebSocket transport mechanism, offering bi-directional communication with low latency, which is particularly important in handling dart hits resulting from detection backend events. This strategy was selected due to its simple implementation and its native support in the TypeScript ecosystem.

In addition to this strategy, considerations were also made in terms of how this communication strategy would behave in an unstable local network state. The communication strategy was modeled in a manner where event data was compact and processing rules were idempotent so that in the event of disconnection or packet loss due to network instability, there was no risk of game state corruption.

Alternative Communication Methods

Alternative communication strategies were also explored using other communication protocols aside from WebSocket transport. Other communication protocols include:

- **HTTP Polling**: This was not selected due to its inherent high latency and unnecessary backend resource usage due to continuous polling.

- **Server-Sent Events (SSE)**: This was not selected due to its limitations in allowing client-server communication without an extra transport layer.
- **WebRTC Data Channels**: This was not selected due to unnecessary added complexity without any significant advantage in this application scenario.

The chosen websocket method further aligns with the system's interaction model, where both event reception (dart hits) and control actions (undo, session transition) need to be represented. This single persistent channel made reasoning and coding easier than with other approaches using two channels.

4.6.3. Architecture Approach: Browser-Based vs. Desktop

Browser-Based Architecture Advantages

Considering that the application is only for desktop, other options, such as using Python GUI libraries (e.g., PyQt, Tkinter) or Java libraries (e.g., JavaFX, Swing), would have been viable. Yet, with the browser-based architecture using SvelteKit, the advantages over other approaches are significant, especially considering the application's specific needs:

- **Cross-Platform Compatibility**: Instantaneous compatibility with Windows, Linux, and macOS without the need for separate installers for each operating system.
- **Deployment Simplicity**: No need to compile or distribute different versions for different operating systems.
- **Visualization Libraries**: Access to state-of-the-art libraries designed for web-based visualizations.

Rationale for Web-Based Selection

The cross-platform compatibility, deployment simplicity, and access to visualization libraries align with the need for cross-platform portability, especially considering the application scenario.

This architecture approach is further advantageous from the perspective of maintainability, especially considering the application scenario. This is because, with the elimination

of platform-specific release processes, the application becomes significantly easier to deploy, especially considering the application scenario. This means that the application's operational process is now centered on server startup scripts, which is significantly easier to implement than client-side releases.

4.6.4. Rendering Technologies

Canvas-Based Game Rendering

The visualization stack leverages HTML5 canvas for game overlay and game state visualization. Canvas is used for its deterministic pixel-based rendering for the virtual dartboard. Canvas is also used for real-time rendering of dynamic game objects like hit points and calibration guides. This is critical for the game interface and meets the accuracy requirements for game score overlay.

From a performance point of view, canvas is chosen for rendering. This is because canvas is event-driven, and the update frequency is dynamic and can spike in certain scenarios like rapid throwing and corrections. Canvas is optimized for direct redraws, and its performance is predictable. DOM/SVG alternatives would introduce recalculation overheads, which are not required for canvas.

Statistical Visualization with Chart.js

Chart.js is chosen for statistics visualization and performance analytics display. It is optimized for common chart types like bar, line, and pie charts. It has minimal configuration requirements and is responsive. These requirements are adequate for the analytical needs. It integrates seamlessly into the SvelteKit frontend stack and has minimal configuration requirements.

Another advantage of using Chart.js is its ability to support incremental feature growth. Currently, only frequency distribution and summary statistics are required. However, the same integration mechanism can be used to support other analytical needs like trend curve plotting over multiple sessions and check frequency distribution among players. This has been done without the need for a heavier visualization stack.

Comparison with Alternative Rendering Approaches

During the initial stages, other rendering technologies were considered for the game application. These alternatives are:

- **SVG-based rendering** (e.g., D3.js): This is useful for applications involving vector-based diagrams. However, for applications involving continuous animation, performance is compromised.
- **WebGL-based rendering** (e.g., PixiJS): This is optimized for high-throughput rendering. However, it comes at the cost of higher complexity, which is not required for game application rendering.

For this project, the combination of HTML5 Canvas and Chart.js provides a good compromise in terms of performance, development effort, and maintainability without compromising on interactivity required in a game scenario.

4.7. Backend and Data Management

4.7.1. Database Selection

SQLite Adoption

The selection process for the database system was influenced by the need for the application to function as a single-user application with local deployment. SQLite met the application's constraints by providing an efficient, lightweight, file-based relational database system with ACID properties, requiring zero administration.

Design Rationale

SQLite does not need process management, unlike other client-server databases (e.g., PostgreSQL, MySQL). The volume of data to be managed by the application is relatively small, with data comprising game sessions, player information, and score events. In addition, using SQLite eliminates the risk of network partitioning or incorrect authentication, common issues with client-server databases, especially during local application evaluation.

For the purpose of this research, SQLite offers the advantage of inspectability. This means that during debugging or verification, it is possible to inspect the data within the database without the need for remote connectivity or specific administration tools.

This characteristic proved advantageous during debugging the statistics module, where defects had to be addressed urgently.

4.7.2. Backend API Selection

FastAPI Framework

The FastAPI framework is used for the backend API due to its capability to support asynchronous API requests, generate OpenAPI schema automatically, and support Python type hints. In this application, the frontend is built using SvelteKit, which offers an integrated web server for serving static assets. In effect, the backend API is only responsible for providing data or event management services.

Architectural Alignment

FastAPI's lightweight architecture, together with its support for asynchronous I/O, is advantageous for the application. This is because, with FastAPI, it is possible for the application's backend to function as an independent process without the need for serving web requests, which is already addressed by SvelteKit. This is particularly advantageous during concurrent WebSocket or API requests during game sessions.

The explicit schema definition via Pydantic within the framework has also been useful for ensuring stability within service contracts for modules written by different contributors. The validation of requests and responses at the API boundary also helped to reduce ambiguity within integrations and improved error reporting by providing structured error messages.

Framework Comparison

Two alternative backend frameworks were considered:

- **Flask**: Provides a similar development experience but lacks native async/await support and suboptimal scaling for concurrent I/O-bound workloads.
- **Django**: A more heavy-duty framework for building full-stack web applications, featuring ORM and template layers, which are unnecessary for building microservice-style API functionality.

Object-Relational Mapping (ORM) Selection

Tortoise ORM has been selected for its native async capabilities and seamless integration with FastAPI. The choice of Tortoise ORM is based on the following reasons for selecting it for this project:

- **SQLAlchemy (Classic)**: This ORM provides a wide range of features but usually requires synchronous blocking calls to work properly, unless extra libraries for async functionality are installed.
- **Django ORM**: This ORM is well-integrated into Django but requires a synchronous approach to requests and responses, making it difficult to use for asynchronous functionality.
- **Tortoise ORM**: This ORM provides a simplified syntax for working with data models and supports native async functionality, matching the asynchronous nature of the project's architecture for WebSockets and RESTful API functionality.

Tortoise ORM allows for all database functionality to be performed asynchronously without any blocking behavior, ensuring responsive gameplay and event handling.

In addition to this, the ORM layer provides enough abstraction to keep the code readable while allowing explicit control over query structure where necessary in performance-critical aggregation paths. This was an important consideration in the statistics pipeline, where simple retrieval operations and grouped summary calculations coexist in a compact code base.

4.7.3. Frontend State Management Technology Decision

Reactive Store Strategy

The state management strategy in the frontend codebase employs native Svelte stores with local storage persistence on selected configuration state. This was determined to be more favorable over using an external state management library due to the compact nature of the domain model and the well-understood interaction graph. The native approach provides direct reactivity, low cognitive overhead, and natural integration with route components.

Persisted vs Ephemeral State

The state management architecture divides state into persisted configuration context (e.g., player selection, selected ruleset, calibration context) and ephemeral state (current throws, turn counters, transient event state). This design avoids potential continuation of stale match state in case of page reloads while improving user convenience in configuration scenarios.

Comparison with Alternative Store Frameworks

Alternative state management strategies using Redux-style state reducers or finite state management libraries were also evaluated in this project. While they offer more formalism in state management, they would have increased implementation and cognitive overhead in this project's context. Native Svelte stores were found to be more than adequate in ensuring deterministic updates while keeping code comprehensibility high enough to accommodate thesis project development.

4.8. Alternative Technology Approaches

4.8.1. Machine Learning-Based Detection

Data Requirements

A learned detector would require a curated dataset of dart impacts across different lighting conditions, board wear states, dart geometries, and camera setups. Building such a dataset at the required scale was outside the scope of this project. Publicly available alternatives were either inaccessible or too narrow to support robust generalization, so a data-driven approach was not selected as the primary method.

Generalization Challenges

Even with sufficient data, model robustness would still need to be validated across sensor resolutions, focal lengths, and lens distortions, all of which change the apparent dart-tip shape. The chosen classical pipeline is less sensitive to appearance variation because it relies primarily on temporal change and geometric constraints rather than learned visual features.

4.8.2. Multi-Camera Systems

Accuracy Improvements

A two-camera setup could triangulate impact position in three dimensions and reduce ambiguity from single-view projection. It would also improve resilience to partial occlusion. However, this additional accuracy is not essential for the recreational scenario targeted in this thesis.

Calibration Overhead

Beyond per-camera intrinsic calibration, a stereo configuration requires stable extrinsic calibration between both cameras. Small mechanical shifts in mounts introduce baseline errors that directly degrade triangulation accuracy. This increases setup complexity and maintenance effort relative to a single-camera installation.

4.9. Technology Selection Rationale

4.9.1. Justification for the Chosen Stack

Alignment with Project Objectives

The selected stack was chosen to satisfy real-time performance, low-cost deployment, and maintainability with minimal hardware assumptions. OpenCV provides the required primitives for calibration, homography, thresholding, and contour analysis, while NumPy enables efficient numeric operations on shared array structures. Python provides rapid iteration and sufficient runtime performance because compute-intensive operations execute in compiled OpenCV routines. ArUco-based calibration supports repeatable setup in fixed installations, and the complete stack runs on Linux and Windows without specialized hardware beyond a standard camera.

5. Implementation

5.1. Data Flow Overview

5.1.1. Architecture Overview

The data flow of the proposed system is structured as a three-tier architecture, where a Python-based detection backend, a database persistence layer, and a SvelteKit-based frontend work collaboratively for real-time dart detection and scoring. This structure allows for the achievement of the proposed objectives of real-time support and data persistence simultaneously.

5.1.2. Input Capture and Detection

Input from a webcam is captured and processed by the detection backend, where OpenCV is used for motion analysis and geometric calibration for dart impact detection and computation of segment scores. The proposed detection pipeline utilizes frame differencing and contour detection techniques for isolating dart impacts from static background elements, where a score is generated for each segment of the dartboard based on coordinate mapping.

5.1.3. Event Emission and Frontend Update

After detecting a valid throw, the backend emits events containing scored segment information through a WebSocket connection, which are asynchronously received by the frontend's game page component. The frontend applies the proposed game logic rules for updating the player's turn, decrementing the scores, and handling special cases such as busts and invalid throws.

5.1.4. Persistence and Data Synchronization

At the same time, processed event data is sent to the database back-end via RESTful API calls to persist data for later analysis. This event-driven design provides fast

feedback to users while ensuring data accuracy and integrity for later analysis. This modular design where detection, logic processing, and persistence layers are separated optimizes system performance.

5.2. Data Model and Persistence

5.2.1. Database Technology Choice

The system architecture fits in with the project constraints as it employs a local data persistence system using SQLite. This design choice was made to avoid implementing a user management system with its own set of complexities in terms of user authentication and potential issues in terms of data latency and transaction problems that could arise in a more complex system.

5.2.2. Core Domain Entities

The data model includes three entities: *Player*, *Game*, and *Score*.

Player Entity

The *Player* entity is characterized by its unique player id and player display name.

Game Entity

The *Game* entity stores player ids participating in a game, initial scoring state, current scoring state, and other game-specific data like creation date and selected game ruleset.

Score Entity

The *Score* entity is used as a transactional entity to associate turn outcomes with player and game data. This data is used as a foundation for later analysis where player performance is evaluated on the basis of aggregated statistics per player per completed game.

5.2.3. Entity Relationship Visualization

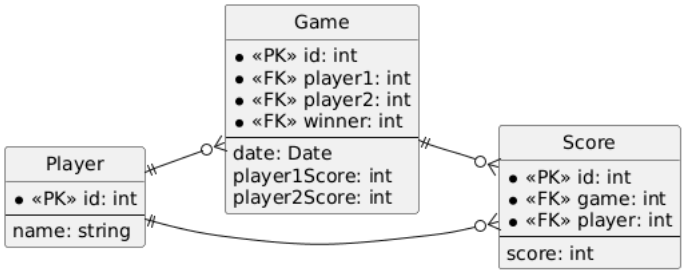


Figure 5.: PlantUML diagram of the data model

5.2.4. Game Rule Modeling

Competitive darts gameplay was constrained to the two conventional rulesets, 301 and 501, because these represent the dominant practice in the targeted application scenario. The selected approach is to normalize the game state by initializing player scores with a ruleset-dependent base value, thereby avoiding structural changes to domain models for each variant.

Listing 5: Game creation endpoint limiting score based on ruleset

```
@router.post("/", response_model=GameDto)
async def create_game(payload: GameCreate):
    base = 301 \
        if payload.version == "301" else 501

    new_game = await Game.create(
        date=datetime.now(),
        player1_id=payload.player1,
        player2_id=payload.player2,
        player1Score=base,
        player2Score=base,
    )
```

Ruleset Selection

The implementation deliberately only supports the 301 and 501 rulesets as explicit options. Less formal variants such as "Around the Clock" were not adopted due to their distinct scoring semantics and their limited relevance to the targeted statistical evaluation objectives.

5.3. Scoring and Game Logic

The scoring logic adheres to established dart scoring rules. During a player's turn, the system decrements the current score by the detected segment value for each throw.

A score of zero constitutes a win condition. If a throw results in a negative value, a *bust* is triggered and the player's score reverts to its value at turn start. Additional rule variants (e.g., "double in" and "double out") are recognized as part of the design considerations, though in the current implementation these are handled by the UI state machine rather than by altering the persistence model.

Logic Responsibility Distribution

Architectural partitioning was motivated by latency and robustness requirements. Core score evaluation and game state transitions are computed client-side in the frontend application, while the backend provides persistent storage and replay-safe data synchronization. This distribution permits immediate user feedback, reduces round-trip delay, and minimizes backend transaction complexity when managing bust and undo interactions.

In this design, all dart and scoring events are persisted regardless of bust status to preserve analytic fidelity, enabling later retrospective computation of per-throw statistics.

Handling Invalid Throws

An invalid throw is defined as a detection event that corresponds to a point outside the calibrated dartboard region. These cases are surfaced in the interface as explicit "miss" actions rather than being ignored, ensuring that the historical event sequence remains complete for metrics extraction. For erroneous detections, the system includes an undo operation that removes the last persisted score entry and re-syncs current game state.

5.4. Calibration

The detection process comprises two stages. The first is to determine the position of the dart tip with respect to the captured frame. The second is to determine the score value from the position of the dart tip, which is mapped to a set of pre-calibrated rings that define the scoring regions of the dartboard. The position of the radial lines dividing the sectors of the dartboard must also be calibrated. The process is analogous to the ring calibration process, which requires user intervention to align the lines between the 20 and 1 sector regions. The manual process is necessary because the geometry of the

dartboard, camera position, and viewing angle differ from one scenario to another. The use of OpenCV was necessary to enable interactive ring calibration in a debug window, which proved to provide a satisfactory level of control responsiveness.

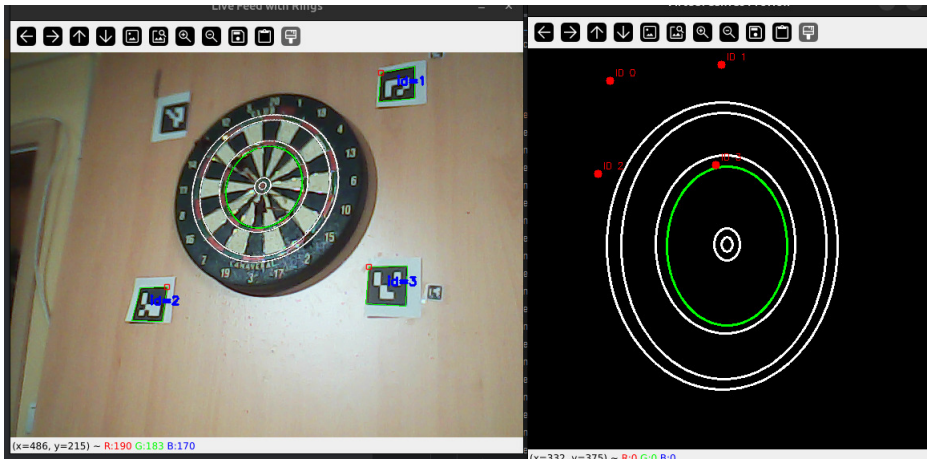


Figure 6.: Early Prototype of the System with the calibration rings drawn on the camera feed.

5.4.1. Frontend-Based Calibration

Camera Resource Management Constraints

The implementation of the frontend-based camera calibration method resulted in technical constraints with respect to camera resource management. The operating system manages the camera resource access at the device driver level, where only a single application process is allowed to maintain an active camera interface connection at any time.

The system architecture allowed for persistent camera resource connection at the Python-based detection backend during application execution. This method did not result in conflicts during normal gameplay, where camera frame notifications using the socket-based method only updated the frontend when the user was engaged with the game view. However, the need for the frontend-based camera calibration method resulted in conflicts with respect to the WebRTC video acquisition method at the detection backend using the Python-based OpenCV detection method.

Sequential Resource Access Solution

The system architecture is modified to implement the sequential resource access method to solve the resource contention problem:

1. The detection backend is initialized without an active camera resource connection.
2. The detection backend waits for the receipt of the calibration data from the frontend.
3. The frontend completes the camera calibration process, whereupon the camera resource is terminated at the frontend level.
4. The detection backend is allowed to access the camera resource for real-time detection.

This temporal separation of camera access between the two components eliminates conflicts while satisfying the functional requirement for both initialization and runtime functionality.

Camera Identification Across Environments

The second challenge encountered during the implementation process resulted from the lack of a standardized camera identification process. This meant that the Python environment, together with the browser, had an independent camera identification process. This process is not standardized, making it impossible to ensure that the user-selected camera index within the WebRTC environment is the same as the camera index within the OpenCV environment. This meant that the user could not be guaranteed to select the correct camera, especially if they are not computer experts. This challenge, however, was mitigated by the assumption that within the single-camera scenario, where only an external camera is connected with the possible addition of the computer's internal camera, both environments would enumerate the cameras in the same order. This meant that the first camera would always correspond to the same device. This assumption provided satisfactory results during the testing phase.

Limitations and User Guidance

The assumption, however, introduced limitations, especially within the multi-camera scenario, where the camera enumeration process would not always follow the same

order. This meant that the user would not always be able to select the correct camera. In order to limit the effect of the assumption, an information box is included within the user interface, providing guidance to the user on the need to consider using the single-camera scenario for optimal results. This is a reasonable solution, considering the single-camera scenario is the only scenario that is likely to be encountered by the target audience.

Coordinate System Definition

The definition and persistence of the coordinate transform from image space to dartboard scoring regions was simplified by the geometric compatibility between the frontend WebRTC video stream and the backend OpenCV frame acquisition: both environments operate on frames with identical pixel dimensions and color spaces. The calibration workflow constructs an HTML5 canvas element dimensioned to match the video frame size, which serves as the rendering surface for interactive ring adjustment overlays. Calibration parameters—specifically the center coordinates (c_x, c_y), scale factor, and anisotropic stretch coefficients (stretch_x, stretch_y) for each concentric ring—are captured through user interaction and serialized to JSON format. This structured data is transmitted to the detection backend via HTTP, where it is persisted to disk as a configuration file. Upon subsequent application launches, the detection backend automatically loads the persisted calibration parameters, eliminating the requirement for recalibration and ensuring calibration consistency across multiple gameplay sessions.

5.4.2. Processing Calibration Data

As previously mentioned, the calibration data is saved in the detection backend as a JSON file.

Listing 6: Example of the calibration data JSON file

```
[
  {
    "cx": 286.0,
    "cy": 238.0,
    "scale": 121.0,
    "stretch_x": 1.0,
    "stretch_y": 1.0
  },
  {
    "cx": 285.0,
    "cy": 235.0,
    "scale": 83.0,
    "stretch_x": 1.0,
    "stretch_y": 1.0
  },
  ...
]
```

Upon backend initialization, the calibration parameters are loaded and used to construct a virtual coordinate grid overlaid on the live camera feed, enabling real-time visualization of the scoring region boundaries as a diagnostic aid during frame processing.

5.5. Dart Detection

5.5.1. Motion Threshold Pipeline

Frame Differencing

The detection backend employs frame subtraction with respect to a blurred background. The current frame is smoothed and compared to the stored background image by absolute difference. The output of this process is thresholded to generate a binary motion mask. This process is the first step in distinguishing between static board information and newly moving dart pixels.

Listing 7: Frame differencing and motion update entry point

```
gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
blurred = cv2.GaussianBlur(gray, (9, 9), 0)

if self.bg_frame is None:
  self.bg_frame = blurred.copy()
  return [], None, [], 0

diff = cv2.absdiff(self.bg_frame, blurred)
thresh = self._apply_filter_pipeline(diff)
```

(a) Input frame (b) Global difference (c) Initial threshold mask

Figure 7.: Frame differencing stage.

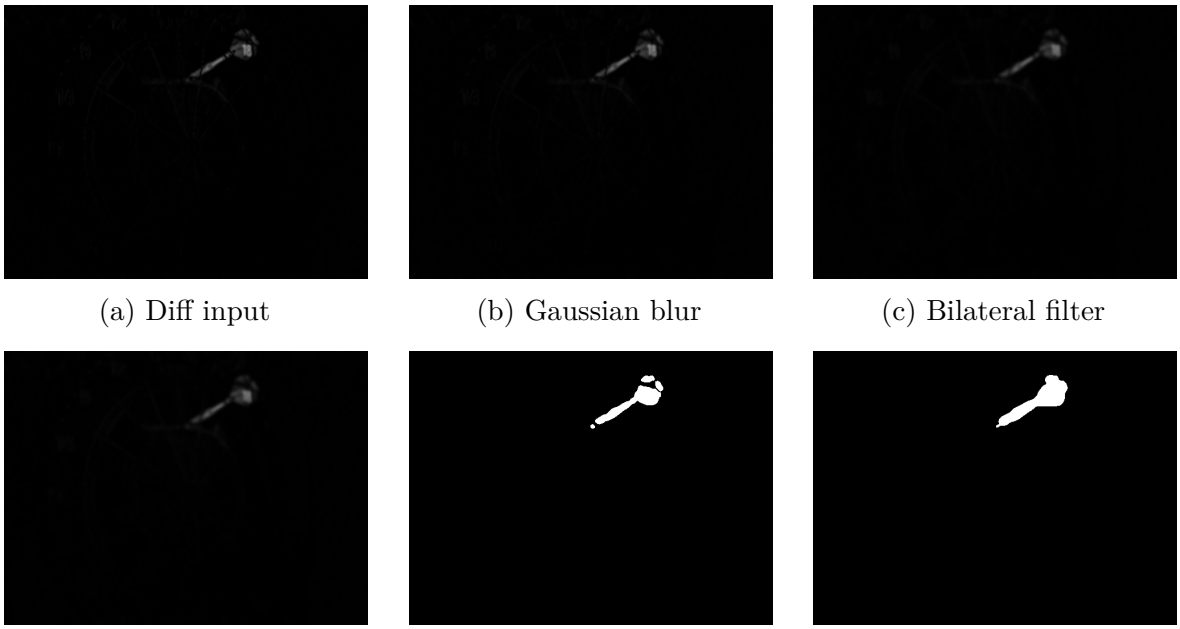
Mask Generation

Subsequent to the differencing step, a fixed thresholding process is employed, where the threshold value can be adjusted using a trackbar. The resulting image is then subjected to morphological refinement, where two dilation and one erosion process take place. This is then followed by the proposed closing process that aims at joining disjointed regions that correspond to the foreground region of interest.

Listing 8: Thresholding and morphology pipeline

```
def _apply_filter_pipeline(self, diff):
  blur = cv2.GaussianBlur(diff, (5, 5), 0)
  for i in range(10):
    blur = cv2.GaussianBlur(blur, (9, 9), 1)
  blur = cv2.bilateralFilter(blur, 9, 75, 75)
  _, thresh = cv2.threshold(blur, self.motion_thresh, 255, 0)
  return thresh

thresh = cv2.dilate(thresh, None, iterations=2)
thresh = cv2.erode(thresh, None, iterations=1)
thresh = self._connect_dart_parts(thresh)
```



(a) Diff input (b) Gaussian blur (c) Bilateral filter

(d) Pre-threshold grayscale (e) Raw threshold (f) Connected mask

Figure 8.: Mask-generation pipeline detail.

5.5.2. Blob Detection and Filtering

Contour Operations

Contours are extracted from the connected binary mask using `RETR_EXTERNAL`, then filtered by minimum area and sorted by contour area. For robust tip estimation, all remaining contour points are merged and converted into one convex hull.

Listing 9: Contour extraction and merge strategy

```
contours, _ = cv2.findContours(thresh, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)
contours = [c for c in contours if cv2.contourArea(c) >= self.min_blob_area]
contours = sorted(contours, key=cv2.contourArea, reverse=True)

all_pts = np.vstack([c.reshape(-1, 2) for c in contours])
all_pts = all_pts.astype(np.float32)
merged_contour = cv2.convexHull(all_pts.reshape(-1, 1, 2))
```

False Positive Reduction

The process also reduces false positives by incorporating several checks, including an ignore mask outside the dartboard ellipse, a suppression area around previously detected dart locations, and a measure of temporal stability based on recent motion history. In cases of excessive global motion, such as camera shake, a reset is performed on the detector, which also updates the background model.

Listing 10: False-positive suppression mechanisms

```
if ignore_mask is not None and mask.any():
  thresh = thresh.copy()
  thresh[mask] = 0

known_mask = np.ones_like(thresh) * 255
for (x, y) in self.known_darts:
  cv2.circle(known_mask, (int(x), int(y)), 12, (0,), -1)
thresh = cv2.bitwise_and(thresh, thresh, mask=known_mask)

if motion_level > 7700:
  self.ready_to_analyze = False
  self.bg_frame = blurred.copy()
  self.known_darts.clear()
  return [], thresh, contour_boxes, motion_level
```

5.5.3. Dart Tip Estimation

Vector Approximation

The ultimate tip estimation is derived by a geometric approximation of the fused contour. To be specific, a convex hull is formed by all the contour points. The leftmost vertex of the convex hull is the candidate tip, with the centroid of the convex hull used as a plausibility reference. A directional criterion is applied to eliminate detections where the candidate is to the right of the centroid, removing a large number of mirrored shapes with this camera configuration.

Listing 11: Tip estimation from merged contour geometry

```
def _estimate_tip(contour, frame_debug=None):
  hull = cv2.convexHull(contour)
  pts = hull.reshape(-1, 2)
  centroid = np.mean(pts, axis=0)
  left_idx = np.argmin(pts[:, 0])
  tip = tuple(pts[left_idx].astype(int))
  return tip, centroid

if tip is not None and centroid is not None:
  if tip[0] >= centroid[0]:
    return [], thresh, contour_boxes, motion_level
```

(a) Largest contour candidates (b) Final tip decision

Figure 9.: Contour filtering and final dart-tip localization.

Line Fitting Techniques

The process of line fitting for the dart shaft was conceptually explored during the implementation process but was eventually dropped. From an implementation perspective, the use of the "merged contour" for the hull estimation was found to provide a more stable outcome under noisy board texture and fragmentation. The final version of the detector thus uses a contour-based fallback, rather than a line model, by selecting the leftmost point of the overall contour stack if the tip fails.

Listing 12: Fallback strategy when hull-based tip extraction is unstable

```
if (tip is None or centroid is None) and isinstance(all_pts, np.ndarray) and
    all_pts.shape[0] > 0:
  left_idx = int(np.argmin(all_pts[:, 0]))
  x = float(all_pts[left_idx, 0].tolist())
  y = float(all_pts[left_idx, 1].tolist())
  tip = (int(x), int(y))
  centroid = np.mean(all_pts, axis=0)
```

5.6. Frontend and Visualization

5.6.1. Technology Stack and Rationale

The need for real-time updates and the logic executed on the frontend prompted us to select SvelteKit, as it compiles to highly optimized native JavaScript with a minimal runtime footprint. Vite was selected as our build tool due to its support for Svelte and hot-module replacement, which was a necessity. Svelte stores with a lightweight add-on for local storage persistence were selected for our state management solution, as our application state is relatively simple and does not justify the complexity of a more sophisticated solution. Our decision to execute the majority of our game logic on the frontend was influenced by the need to respond immediately to user and socket-related events, as well as the desire to minimize backend load.

The chosen stack also supports the practical deployment model of this thesis: one local machine running both detection and application services, and one or more client devices connected via local network. In this context, startup speed, low memory overhead, and predictable update behavior were valued more than advanced enterprise features. SvelteKit with Vite satisfied this profile by allowing short development iteration loops and by producing a compact production bundle that can be served with minimal configuration effort.

Another important architectural criterion was deterministic behavior under frequent asynchronous events. During active play, websocket hit events, undo interactions, view transitions, and persistence requests can overlap in short intervals. The selected frontend architecture supports this workload through explicit store updates and isolated side-effect boundaries in page-level modules, which simplifies debugging and reduces hidden state coupling.

5.6.2. Application Module Structure

The codebase for our frontend application mirrors modern best practices for a SvelteKit-based project. The application entry point is the `src/routes/` directory, which holds all page-related code. Each significant user interface, such as the game, setup, or statistics screen, is represented by a separate subdirectory within the `routes/` directory. For instance, the game-related code can be found within the `routes/game/` directory, while the setup-related code is contained within the `routes/setup/` directory. These directories house Svelte components that represent each of our user interface views.

Reusable UI components such as buttons, headers, or step counters are stored within the `src/comps/` directory. State management logic, represented by Svelte stores, is stored within the `src/stores/` directory. The calibration module is encapsulated in a specific component (`CameraCalibrator.svelte`) and store. This allows for interactive camera configuration and persistence of the camera's parameters. Although the socket client and service components are encapsulated in the respective page components and stores, the design allows for real-time synchronization of game states and statistics through event-driven updates.

From a maintainability perspective, the structure follows two explicit rules. First, rendering components should remain largely declarative and avoid embedding non-trivial game logic. Second, mutation of persistent or globally shared state should occur through dedicated store actions rather than direct ad hoc writes. This split improved testability and reduced regressions when features such as undo behavior and per-player statistics were added later in the project.

The modular arrangement also supports progressive extension of the system. New game variants or additional analytics widgets can be introduced by extending store-level selectors and dedicated route components without rewriting the existing transport and synchronization layer. This is particularly important in a thesis prototype because it enables demonstrable extensibility while preserving implementation clarity.

5.6.3. Routing and Navigation Flow

The frontend follows a hierarchical routing structure that presents users with a specific navigation flow. The application's root route (`(/)`) is used to display the initial configuration page. Users configure their game by selecting the ruleset (either 301 or 501), as well as the play variant (standard or alternate), which is persisted in the application's state using the reactive store `rulesState`. Users navigate to route (`(/setup)`), which allows users to select the two participating players. Users can search and select the two players in the game. The player information is persisted in the application's state using the reactive store `playerState`. Users navigate to route (`(/setup/camera)`), which allows users to configure the camera's parameters. Users can interactively configure the camera's parameters and then asynchronously persist the configuration in the detection backend. Users navigate to route (`(/game)`), which allows users to view the game's interface. Users retrieve player information and game ruleset information from the application's state using reactive stores. Users then connect to the

game server via a WebSocket and perform real-time dart event handling. Users navigate to route (`(/stats)`), which allows users to view player analytics and game statistics. Users can query player-specific information. This structure also provides consistency, as all reactive stores, i.e., `playerState`, `rulesState`, and `calibrationStore`, are being subscribed at application initialization.

Route gating was used to enforce a valid setup order. If required prerequisites are missing (e.g., player identities not selected), the route logic redirects users to the corresponding setup page. This prevents invalid game starts and ensures that gameplay pages always receive a coherent initialization context. In practical usage, this reduced operator confusion and lowered the number of state-repair actions needed during demonstrations.

Listing 13: Frontend route hierarchy and state flow

```
// Application route structure implemented via SvelteKit conventions
/ (root)
  +page.svelte          // Ruleset selection (301/501) -> rulesState store
  +layout.svelte        // Global layout & store subscriptions
  /setup
    +page.svelte        // Player selection -> playerState store
    /camera
      +page.svelte      // Camera calibration -> calibrationStore
  /game
    +page.svelte        // Gameplay orchestration (socket, rules, rendering)
  /stats
    +page.svelte        // Win percentage, hit metrics, score distribution
```

5.6.4. Rendering Strategy

The rendering of the virtual dartboard and its interactive overlays is accomplished using the browser's native HTML5 canvas API, orchestrated from within Svelte components. Overlays such as sector boundaries, calibration rings, and throw markers are drawn programmatically, enabling dynamic updates in response to game events. Throw markers are rendered as layered graphical primitives (e.g., circles, lines, and text) on top of the static board image, with their positions determined by the coordinates passed from the socket. This approach was chosen over alternatives such as SVG or DOM-based rendering due to the superior performance and lower latency of canvas for frequent, real-time updates, as well as its ability to efficiently handle complex, pixel-level manipulations required for accurate hit visualization.

The rendering pipeline was designed with a strict separation between domain data and drawing primitives. Score events are transformed into a normalized internal marker representation before any canvas operation is performed. As a result, visual style adjustments (color, marker shape, label density) can be implemented without changing

synchronization logic or score computation. This separation was beneficial when tuning readability for different screen sizes.

Coordinate Mapping Between View and Backend

To ensure accurate mapping between backend-reported hit locations and frontend visual feedback the frontend implements a coordinate transformation pipeline. Canvas coordinates, which are inherently dependent on the current viewport size and device pixel ratio, are normalized to a canonical board coordinate system. Calibration parameters, obtained during the setup phase, define the geometric relationship between the camera-captured board and the rendered canvas, allowing for mapping between frontend and backend coordinate spaces.

To reduce drift between coordinate spaces, the mapping implementation performs scaling and offset transformations in a fixed order and stores precomputed factors where possible. This avoids repeated floating-point recomputation on each render cycle and lowers the risk of frame-to-frame marker jitter. Additionally, the implementation distinguishes between display coordinates and scoring coordinates, so visual interpolation effects do not modify the authoritative score event payload.

Visual Feedback and Error States

There is also immediate visual feedback in the user interface. For a valid throw, there is a change in the scoreboard according to the points scored, and there is a visual marker on the dartboard. For an invalid throw, there is an X marked on the scoreboard. In this case, the virtual dartboard has limitations, so an out-of-bounds throw is not shown on the dartboard; it is shown just as a missed throw. All this is to ensure maximum awareness, minimum incorrect user actions, and a seamless interface in all states.

Table 9.: UI feedback states.

State	**Current UI Behavior**
Valid hit	Shows score update and visual marker
Bust	Clears Scores out of Scoreboard
Miss / out-of-bounds	Shows miss on Scoreboard

5.6.5. Game UI

Real-Time State Synchronization

Real-time synchronization of game state across multiple clients is achieved through WebSocket-based event streams. The frontend establishes a persistent WebSocket connection to the backend and subscribes to dart-hit events. Upon receiving an event (e.g., a `"dart hit", "T20"` message), the client applies the appropriate ruleset logic and updates local state.

Operationally, synchronization reliability depends on deterministic event ordering and robust duplicate handling. The frontend therefore treats each incoming event as an immutable state transition input and appends it to a per-turn history structure before derived UI values are updated. This ensures that features such as undo, turn rollback after bust, and post-game statistics all reference a consistent historical sequence rather than transient rendered values.

For multi-client viewing, local rendering remains optimistic while persistence confirmation is executed asynchronously. If an API write fails, the interface marks the affected event and exposes a retry path instead of silently discarding the discrepancy. This approach preserves user trust because mismatches are made explicit and can be corrected without restarting the session.

Scoreboard Interaction Model

The scoreboard interface is designed to provide intuitive and immediate feedback for all core game actions. Turn progression is visually indicated, with the active player and remaining throws clearly highlighted. An Undo button allows users to correct falsely registered throws. All interactions are tightly coupled to the underlying state management logic, ensuring that the UI remains synchronized with the authoritative game state.

Listing 14: Frontend event/state flow

```
websocket.on("dart_hit") => {
  subtract score from current player;
  apply game rule logic;
  persist throw;
  update local state;
  render scoreboard and dartboard ;
}
```

5.6.6. Game Logic and State Management

Ruleset Engine (301, 501, Double Out)

The frontend implements the core rulesets for standard dart games, including 301, 501, and Double Out, directly within the application logic. Each ruleset is encapsulated as a set of functions that evaluate throw outcomes, manage score reductions, and enforce bust and win conditions. Busts are detected by simulating the score after each throw and reverting the state if the player's score would fall below zero or violate the ruleset's constraints (e.g., failing to finish on a double in Double Out mode). Winning conditions are checked after each valid throw, with the UI providing immediate feedback when a player achieves a legal checkout.

Turn Lifecycle and Throw Sequencing

Turn management is orchestrated through a combination of state variables and event-driven transitions. Each turn begins with the activation of a new player and the reset of the throw counter. Throws are tracked sequentially, and the turn concludes either after the maximum number of throws is reached or when a bust or win condition is triggered.

The sequence controller applies explicit transition guards to prevent illegal transitions such as processing a fourth throw in a completed turn, accepting new throws after a terminal win, or switching players before bust rollback is applied. This finite-state style design made behavior easier to reason about and significantly reduced defects during integration testing with live websocket events.

In addition, each turn stores both the starting score and the per-throw deltas. This allows deterministic reconstruction of every intermediate state and simplifies both undo and audit views. The same representation is reused by the statistics module to compute phase-level metrics (early-turn vs checkout behavior) without re-querying backend game snapshots.

State Consistency Across Views

The state consistency across different views (e.g., setup, statistics, game) is achieved by centralizing all authoritative game data under Svelte stores, which are automatically reactive and accessible across the application. There is no need to re-fetch data or re-initialize state when moving between views, as the current state is automatically

preserved and propagated, thus avoiding data artifacts and providing a seamless user experience. Any changes to the underlying data model will automatically reflect across all affected UI views, thus supporting robust multi-view scenarios.

Store hydration and cleanup policies were introduced to prevent stale state propagation between independent sessions. At game start, stores are initialized from validated setup data only. At game completion or explicit reset, transient gameplay stores are cleared while long-lived configuration stores (players, calibration profile) are retained. This selective persistence strategy improves usability by keeping high-value setup data while eliminating state leakage into subsequent matches.

5.6.7. Statistics and Persistence

Data Model for Match History

The persistence layer is achieved through a relational schema with three main entities: `Player`, `Game`, and `Score`. The schema is normalized to enable efficient query operations on player-specific statistical data through indexing on the `player_id` field. The hierarchical structure of the entities (Player ← Game ← Score) is advantageous when performing aggregation queries without having to perform costly joins on tables. The frontend interacts with the entities through REST calls that return filtered score sets for a given player, enabling computation of derived values on the client-side.

Statistics Computation Pipeline

The computation of statistical values is spread across frontend and backend code based on complexity and latency constraints. The backend computation of statistical values includes the computation of "win percentage" through aggregate queries on the `Game` entity. This is computed as the ratio of instances where `winner_id` equals the player identifier and instances where the player participated as `player1` or `player2`. The computation of statistical values on the backend avoids redundant iteration over game objects and leverages the efficiency of database indexing. The computation of derivative values from the received sequence of scores is performed on the frontend. This includes computation of hit percentage (ratio of non-zero values to total throws), statistical mode (most frequently occurring non-zero value of scores), and cumulative point total and throws. This frontend computation avoids transmitting redundant data (only

transmitting values of scores instead of game objects) and allows for immediate response to user interactions without requiring server round-trips.

The split computation model was selected deliberately to keep network payloads compact and to retain interactive responsiveness in the analytics view. Metrics that rely on relational joins and game ownership semantics are computed server-side, while metrics that are simple folds over already fetched throw sequences are computed client-side. This partitioning lowered backend complexity and allowed instant recalculation when users switched between players in the UI.

Statistics Views and Interpretation

The interface has five main metrics that are aggregated from the match history of the player. The *Darts Thrown* metric represents the total number of recorded throw events for the player. The *Total Score* metric represents the total number of points scored by the player. The *Hit Percentage* metric represents the percentage of successful throws for the player. It is calculated as non-zero throws divided by total throws multiplied by one hundred. The *Most Common Hit* metric represents the most frequently scored non-zero hit value for the player. The *Win Percentage* metric represents the percentage of games won for the player. It is calculated as games won divided by total games played multiplied by one hundred. There is also a bar chart representing the frequency distribution of the ten most common non-zero scores for the player.

Error Handling for Missing/Corrupt Historical Data

The computation of statistical values has been equipped with defensive programming strategies that ensure graceful degradation when faced with situations involving incomplete or absent historical data. The client code has been designed to recognize when a set of scores is absent for a selected player by detecting that the score collection is falsy. In this situation, instead of propagating NaN values to the user interface, it displays a placeholder that represents the absence of historical data. The division by zero defensive programming strategy is used when computing the win percentage. The code has been designed to return a sentinel value of 0.0 when a player has zero games played instead of throwing a division by zero exception. The computation of the most common score is performed when there is a non-zero value within the set of scores. If all throws by a player are misses, then this computation is bypassed, ensuring that the initialized value is not propagated to the user interface. The Chart.js instance that is

used to display the score distribution is deliberately destroyed and recreated each time the metric values are refreshed to ensure that stale data is not displayed when changing players.

Beyond these defensive checks, basic data provenance validation is performed before rendering analytics cards. Payloads are validated for required keys and type consistency; malformed records are excluded from aggregation and reported through a non-blocking warning state in the UI. This strategy avoids complete view failure due to isolated corrupt records while preserving transparency regarding data quality.

Listing 15: Statistics computation exemplar showing metric derivation from persistent score records

```
// Backend: win percentage calculation via aggregate counting
wins = COUNT(Game WHERE winner_id = player_id)
total_games = COUNT(Game WHERE player1_id = player_id OR player2_id = player_id)
win_percentage = wins / total_games * 100

// Frontend: metrics derived from fetched score array
let scores = await fetch("/score/{player_id}").json()
let nonZero = scores.filter(s => s.score > 0)
let hitPercentage = (nonZero.length / scores.length) * 100
let totalScore = scores.reduce((sum, s) => sum + s.score, 0)
let dartsThrown = scores.length
let mostCommonScore = MODE(nonZero.map(s => s.score))
```

5.6.8. Operational Reliability in Frontend Workflows

Connection Lifecycle Management

Reliable websocket operation requires explicit handling of temporary disconnects, route changes, and backend restarts. The frontend therefore maintains a bounded reconnect policy with incremental delay, coupled with connection status indicators in the gameplay view. When reconnection succeeds, the client re-synchronizes the active game context before accepting new throw events. This prevents event ingestion under ambiguous session state and lowers the risk of score divergence.

Undo and Correction Workflow Integrity

Undo functionality was treated as a first-class reliability feature because detection errors are unavoidable in practical environments. The workflow follows three ordered steps: remove the most recent authoritative throw from local history, propagate deletion to persistence storage, and then recompute derived game state from the revised throw sequence. This replay-based recomputation is computationally inexpensive for the

targeted data scale and avoids subtle arithmetic drift that can occur when manually patching multiple dependent counters.

Usability Under Shared-Device Conditions

The system was explicitly designed for usage scenarios in which players interact from different devices on the same local network. To support this, interface actions that can alter authoritative state (start game, undo throw, end session) are visually distinguished from passive navigation actions. Combined with explicit turn highlighting and throw counters, this reduces accidental state changes in social play environments where multiple users observe and occasionally interact with the system concurrently.

Frontend Contribution to System-Level Robustness

Although detection quality is primarily influenced by computer-vision design, frontend behavior materially affects perceived reliability. Immediate visualization of accepted throws, explicit representation of misses, and clear handling of error conditions allow users to detect and correct inconsistencies quickly. In this way, frontend architecture is not only a presentation layer but also a practical control and verification interface that stabilizes end-to-end system operation.

5.7. Hardware Design

5.7.1. Backboard Construction

Material Selection

Hardware prototype, which is movable, necessitated the design of a support structure for the dartboard and the camera. A wooden frame was thus adopted. The rationale behind this design choice is the availability, cost-effectiveness, and stiffness of the wooden frame to guarantee the relative position of the dartboard and the camera upon repeated assembly and disassembly.

For a stationary and permanent installation, the support structure is not needed. The dartboard is fixed when installed on the wall, just like the camera. The support structure is thus needed to make the hardware prototype portable.

5.7.2. Camera Mounting

Two methods of camera mounting were considered. The first method is a fixed mount, which is fixed to the frame or a wall. The second method is a tripod mount. This allows faster deployment without having to design a custom mount.

Regardless of which mounting strategy is used, an essential requirement is to ensure that the dartboard is fully contained within the captured image. The camera should be positioned to maximize coverage of the dartboard within the image. The board should not be cut off at the top or bottom. If the distance is too great, making the board too small within the image can cause difficulties in resolving details on the tip of the dart.

Angle Optimization

Optimization of the angle selection process was carried out through empirical evaluation. It was observed that at very large lateral angles, the contour of the dart shaft tends to be more readily discernible; however, the regions at the opposite end of the board tend to be more compressed in classification reliability. Conversely, if the camera were oriented nearly directly at the board, the sector geometry tends to be more interpretable; however, the contour information from the shaft side tends to be weaker.

It was observed through experimental results that the most balanced approach for the camera perspective appears to be at an intermediate 45° orientation between the two extremes. When using low-resolution camera systems in which tip detection tends to be unstable, tilting the camera further toward the board at an angle of 50° to 60° appears to increase the robustness of the system, in which the end of the dart shaft tends to be more readily utilized as the terminal end for more stable performance.

5.8. System Integration

5.8.1. Hardware-Software Interaction

Pipeline Latency

System integration links three independently developed modules: image-based detection, game-state orchestration, and persistence/statistics services. The integration objective was not only functional interoperability but also predictable latency under realistic

gameplay pacing. The event path starts with a detected hit in the vision backend, continues through websocket transport to the frontend game logic, and ends with asynchronous persistence and UI refresh.

From the perspective of user experience, the decisive metric is perceived responsiveness, defined as the delay between physical dart impact and visible score update. The architecture minimizes this delay by applying score transitions immediately on websocket reception, while persistence writes execute in parallel. In practice, this design yielded responsive interaction even when API operations were temporarily slower than nominal.

To keep module boundaries explicit, integration contracts were reduced to stable payload structures: event type, scored segment value, optional coordinate metadata, and game/session context identifiers. Restricting the interface surface in this way simplified debugging and reduced the probability of cross-module regressions when internal implementations evolved during iterative development.

Failure Isolation Between Modules

A core integration principle was failure isolation. If one subsystem enters a degraded state, the remaining subsystems should continue operating as far as possible:

- If persistence requests fail, the gameplay view remains active and marks affected events for retry.
- If websocket transport is interrupted, the UI enters a controlled waiting state and blocks further authoritative transitions.
- If frontend rendering fails for a non-critical visualization widget, score progression still follows the authoritative state store.

This isolation approach was essential for practical testing because it transformed critical faults into recoverable states instead of forcing full application restarts.

Observability and Diagnostics

Integration debugging was supported by lightweight runtime observability: timestamped event logs, explicit connection status, and structured error labels for transport, logic, and persistence layers. These diagnostics enabled quick attribution of anomalies to

the responsible subsystem and reduced ambiguity during collaborative development between authors.

5.8.2. System Limitations

Environmental Dependencies

6. Evaluation and Testing

6.1. Test Environment

6.1.1. Lighting Conditions

Challenges Identified

Lighting was recognized as a critical factor to ensure proper dart detection. Early experiments identified how shadows cast by dart shafts and the person throwing created unstable contour shapes and periodic false motion areas during the frame differencing stage.

Solution: Ring Light Implementation

The lighting problem was addressed by adding a ring light positioned around the dartboard. This modification provided two key improvements:

- More uniform light distribution on the dartboard surface
- Significant reduction in directional shadows from dart shafts and surroundings

Impact on Detection Stability

Under the new lighting condition, contour detection was noticeably more stable during successive throws, especially in critical areas near ring boundaries where small variations in lighting would normally cause significant variations in tip position estimation.

Remaining Error Sources

Errors observed under the optimized lighting condition were mainly attributable to calibration issues rather than lighting-related factors, confirming that the ring light approach successfully mitigated the primary lighting challenges. This demonstrates that with proper illumination, other system constraints (primarily calibration accuracy) become the limiting factor.

6.1.2. Camera Setup

Primary Camera Configuration

All evaluation runs were performed with a standardized camera configuration to enable reproducibility and consistency across test sessions:

- Webcam model: UGREEN Camera 4K
- Resolution: 1920×1080 (FHD)
- Effective frame rate during testing: 26 FPS
- Stream mode: RGB video

Spatial Setup Parameters

The field setup parameters were carefully chosen to balance image coverage with pixel-level detail:

- Camera angle: approximately 50°
- Camera distance: approximately 60 cm

Image Characteristics

Additional camera characteristics that were noted during the setup process:

- Image size: 2.07 MP
- Aspect ratio: 1.78
- Average brightness level: approximately 46%
- Saturation level: 2.59%

Distance and Coverage Rationale

The camera distance was not based on any specific geometric constraint but rather on practical considerations: the ability of the image to contain the entire dartboard while simultaneously maximizing the board occupancy in the frame. This choice is justified since increasing camera distance reduces the pixel detail at the dart tip contour, which directly impacts localization accuracy. The chosen 60 cm distance represents an optimal

balance between complete board visibility and sufficient pixel-level resolution for tip detection.

Table 10.: Evaluation setup sheet used for repeatable test sessions.

Parameter	**Configured Value**
Camera model	UGREEN Camera 4K
Resolution	1920×1080
Effective frame rate	26 FPS
Camera angle	approximately 50°
Camera distance	approximately 60 cm
Illumination strategy	Ring light around board to suppress directional shadows
Primary evaluation goal	Stable sector classification under practical throw scenarios

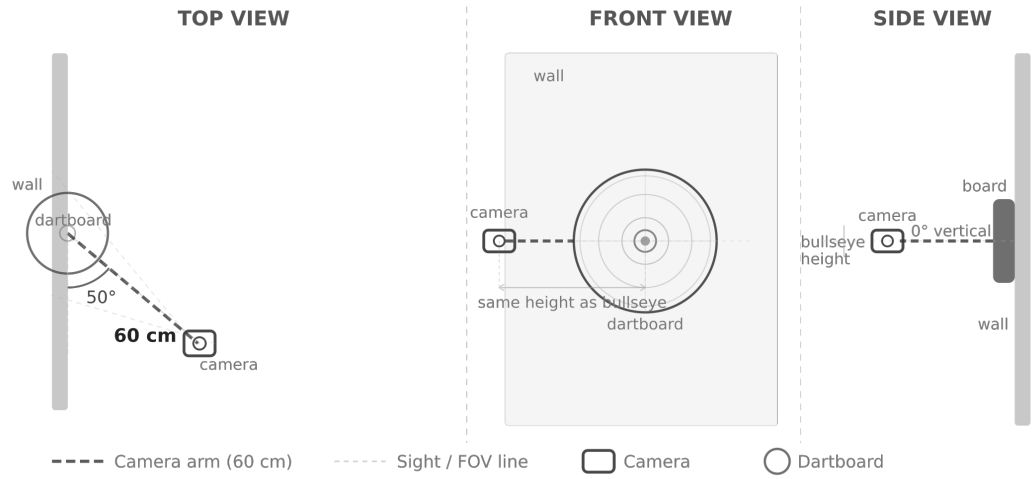


Figure 10.: Conceptual sketch of the evaluation setup geometry (camera angle and distance).

6.2. Evaluation Metrics

6.2.1. Hit Position Error

Localization Accuracy Approach

The system maps the identified tip coordinate location onto the ring sector region in order to classify the hit location. Rather than using millimeter-level error metrics, the evaluation focused on the ability of the localization system to correctly classify the sector region, which is the metric that directly impacts gameplay accuracy.

Central and Mid-Sector Performance

Qualitatively, the robustness of the system was observed in the impacts made at the center and the mid-sector regions, where the contour is most stable and clearly separated from boundaries.

Boundary-Region Uncertainty

The most prominent localization uncertainty was observed in near-boundary throws, where minor contour shifts can cause the tip location to switch from one sector region to another. This behavior can be attributed to the single-camera projection system, where minor distortions and contour variations have the most significant impact near the decision boundaries. The single-view perspective inherently introduces ambiguity at region edges.

6.2.2. Sector Classification Accuracy

Overall Classification Performance

Across extensive practical throwing sessions (including central hits, boundary-near hits, and out-of-board throws), sector-level classification remained stable enough for recreational play. The quantified accuracy values are reported in Section 6.3, where setup-specific outcomes and detailed accuracy calculations are provided.

Misclassification Categories

Analysis of misclassifications identified three primary failure modes:

- **Calibration Issues**: Calibration drift or poor calibration quality, causing systematic sector offset across all throws.
- **Boundary Ambiguity**: Boundary ambiguity when a large contour intersects a sector edge, producing unstable assignment between adjacent sectors.
- **Mechanical Vibration**: Vibration-related global motion when the camera mount couples mechanically to the dartboard.

Error Proximity Analysis

The first two categories provided results that, although incorrect, were reasonable within a neighboring area rather than arbitrary data, which further reinforces the proximity of the localization to the actual impact location. This indicates that the underlying detection and localization pipeline is fundamentally sound, with errors occurring near decision boundaries rather than systematic offsets across the board.

Table 11.: Observed practical outcome classes during evaluation sessions.

Outcome Class	Relative Frequency	Typical Cause
Correct sector assignment	High	Stable calibration and clean contour extraction
Neighbor-sector misclassification	Medium	Boundary ambiguity and contour edge expansion
Missed hit event	Low to Medium	Camera vibration or insufficient stable post-impact contour
Correctly ignored non-hit event	Medium	Out-of-bounds throw or no persistent board impact

6.3. Accuracy Consolidation

Accuracy is reported at sector level and is based on labeled throws from the evaluation session described in Table 10. Camera placement constraints that influence classification stability are discussed in Section 6.1.2.

For each setup, accuracy is calculated as:

$$\text{Accuracy} = \frac{N_{\text{correct}}}{N_{\text{total}}} \cdot 100\%$$

6.3.1. Setup A: Measured Outcomes

Table 12.: Outcome counts for Setup A.

Outcome Category	**Count**	**Interpretation**
Correctly detected	43	Correct sector classification
Wrong edge cases	7	Dart tip not correctly localized or poorly calibrated
Occluded by another dart	2	Visibility limitation due to overlap
Not detected because of motion	3	Motion-related detection failure
Total throws	**55**	Sum of all outcome categories

Computed real accuracy (Setup A):

$$\frac{43}{55} \cdot 100\% = 78.2\%$$

6.3.2. Setup B: Alternative Setup

Table 13.: Setup B specification sheet.

Parameter	**Value**
Camera model	TRUST EXIS Webcam
Resolution	640×480
FPS during test	24
Camera angle	45°
Camera distance	65 cm
Lighting	Same ring light

Table 14.: Outcome counts for Setup B.

Outcome Category	Count	Interpretation
Correctly detected	29	Correct sector classification
Wrong edge cases	7	Dart tip not detected due to limited resolution
Occluded by another dart	2	Visibility limitation due to overlap
Not detected because of motion	10	Camera mount too unstable
Total throws	**48**	Sum of all outcome categories

Computed real accuracy (Setup B):

$$\frac{29}{48} \cdot 100\% = 60.4\%$$

Table 15.: Overall accuracy summary across both setups.

Scope	Total Throws	Correct	Computed Accuracy (%)
Setup A	55	43	78.2
Setup B	48	29	60.4
Combined	**103**	**72**	**70**

6.4. Frontend, State, and Analytics Evaluation

6.4.1. Real-Time Interface Responsiveness

Measurement Procedure

In addition to the accuracy of the vision system, real-world usability is affected by the time it takes for the interface to update the score once a hit is detected. To assess the real-time responsiveness of the system, multiple cycles of game play sequences were performed, and the time from event emission in the backend to state transition in the frontend and finally to the score update in the interface was monitored. The evaluation did not aim to precisely measure time delays, as this is not critical to game play. Instead,

the evaluation aimed to assess the consistency of the interface in updating the score in real time.

Observed Behavior

During the evaluation, the score update in the interface was observed to be immediate for regular play. For game play involving rapid sequential throws, the interface behaved correctly, updating player status, score, and number of throws in the expected order. This is a real-world confirmation of the websocket and store combination providing real-time score feedback as required for a game play interface.

Degraded Conditions

To further assess the interface and game play, the backend server was restarted, and the websocket connection to the frontend interface was terminated. The interface correctly disconnected from the game play state and reconnected once the websocket connection was re-established. This is relevant to game play, as it confirms the interface and game play state correctly revert to a disconnected state in response to a lost websocket connection, without affecting the score. This is important, as it shows the game play state is not corrupted when the interface re-establishes the websocket connection and sends a request for the latest game state, which is not available at the time of reconnection.

6.4.2. Game State, State Transitions, and Analytics Evaluation

Turn and Bust Validation

To assess the state transition and game logic, specific game play sequences involving busts, checkouts, and turn rollover from three throws to a new turn were performed.

Undo Workflow Verification

Undo functionality was verified with normal and boundary condition scenarios, for example, undo immediately after bust and undo after route change. In all cases, the implementation correctly removed the latest throw, recalculated the values, and refreshed the views. This verifies the reliability of the replay-based recalculation for the chosen domain size.

Cross-View State Continuity

Switching routes between setup, game, and statistics views was performed several times for active and finished sessions. The state was always consistent without requiring a full page reload. This verifies the store implementation and confirms one of the primary frontend verification goals: ensuring game continuity for navigating to analytics and configuration views.

Table 16.: Verification scenarios for the backend implementation with the observed results.

Scenario	Expected Behavior	Observed Result
Three valid throws in one turn	Automatic turn handover after third throw	Confirmed in multiple sessions
Bust condition triggered	Score reverts to turn-start value	Confirmed for various sequences
Undo after valid hit	Last throw removed and UI recalculated	Confirmed with consistent state replay
Temporary websocket disconnect	Controlled waiting state and safe resume	Confirmed with short-term disconnections
Switch to statistics and return to game	No loss of active session context	Confirmed for route transitions

6.4.3. Multi-Client Synchronization Behavior

Local Network Viewing

The architecture also allows multiple clients on a local network to share in the observation of the state changes in a game. During testing with two simultaneous browser clients, it was observed that both views correctly reflected incoming hit events in nearly identical order with no apparent desynchronization in turn ownership or score values.

Consistency Safeguards

The consistency was ensured through handling websocket events as ordered state transition inputs and ensuring unsynchronized local state changes were not made. The update discipline on the store and action boundaries was found to be effective in this

case scenario. Thus, the user interface was found to behave correctly even in cases where one client lost focus or was subject to minor rendering delays.

6.4.4. Statistics and Persistence Validation

Metric Correctness Checks

The statistics view was found to behave correctly through comparisons with manually computed values using selected persisted score sequences. The metrics were found to be accurate in cases where darts thrown, total score, hit percentage, and win percentage were computed using non-zero values in the selected score sequences.

Handling Sparse and Empty Data

New player instances with sparse data were used to test the analytics view's handling of sparse data. The UI was found to behave correctly in this case scenario, presenting stable values instead of undefined or not a number values in cases where data was sparse or missing. This shows that the code correctly implements guard clauses against presentation-level failure in cases where data is missing or sparse.

Interpretability of Statistical Views

From an operational viewpoint, the chosen metric set has been found to be adequate for rapid interpretation following a game session by non-technical players. Players can easily identify immediate trends, such as having a low hit percentage despite high throw volume, without the need for external tooling. This validates the thesis's objective of providing accessible and meaningful feedback in conjunction with automated scoring.

Listing 16: Practical validation protocol for frontend and statistics behavior

```
1 Start game with known ruleset and two predefined players.
2 Determineistic throw sequence with expected score computation.
3 Bust and undo paths are triggered deliberately.
4 Score state is compared to manual step-by-step computation.
5 Statistics view is opened and compared to manual computation.
6 Repeat for player with empty history to check for proper placeholder display.
```

6.4.5. Summary of Frontend Evaluation Findings

The frontend has been found to be adequate for its practical purposes in the tested environment. Updates are responsive in real-time, state computation is deterministic in

most common correction scenarios, and analytics computation matches expected values for representative datasets. However, to be fully robust in hostile network environments, further hardening is necessary. For the purposes of the thesis, the current state is adequate for the proposed home and laboratory environment.

6.5. Edge Case Analysis

6.5.1. Bounce-Outs

It is not possible to detect these with the current pipeline. The detector assumes that a valid throw results in a newly static object on the board. If a dart bounces off a metal area or touches a dart and then is thrown off, then no final static contour exists, so it is not considered a valid dart.

This is a known methodological limitation as opposed to a programming error. The development of a proper bounce-out detector would require further event modeling, sound/vibration detection, or multi-angle verification.

6.5.2. Occluded Darts

Extreme cases of occlusion were rare in the given board configuration but are relevant in practical environments. If a new dart is thrown into an area close to an already embedded dart, contour union can reduce dart-tip separability. The merged-contour technique currently used is a significant enhancement to increase robustness against fragmented dart contours. However, it can also cause ambiguity between densely grouped darts.

From an empirical standpoint, it was determined that these types of situations were most likely to cause issues with shots landing in near-adjacent segment areas rapidly fired in succession.

6.5.3. Motion Blur and Shadows

Given a 26 FPS rate and ring light illumination, motion blur was not a significant factor in error rates, as the final state is stationary to register a point. However, sudden changes to global illumination or cast shadows can potentially cause issues with threshold determination.

The ring light installation was a significant factor in minimizing potential issues. The few errors that occurred were due to camera vibrations.

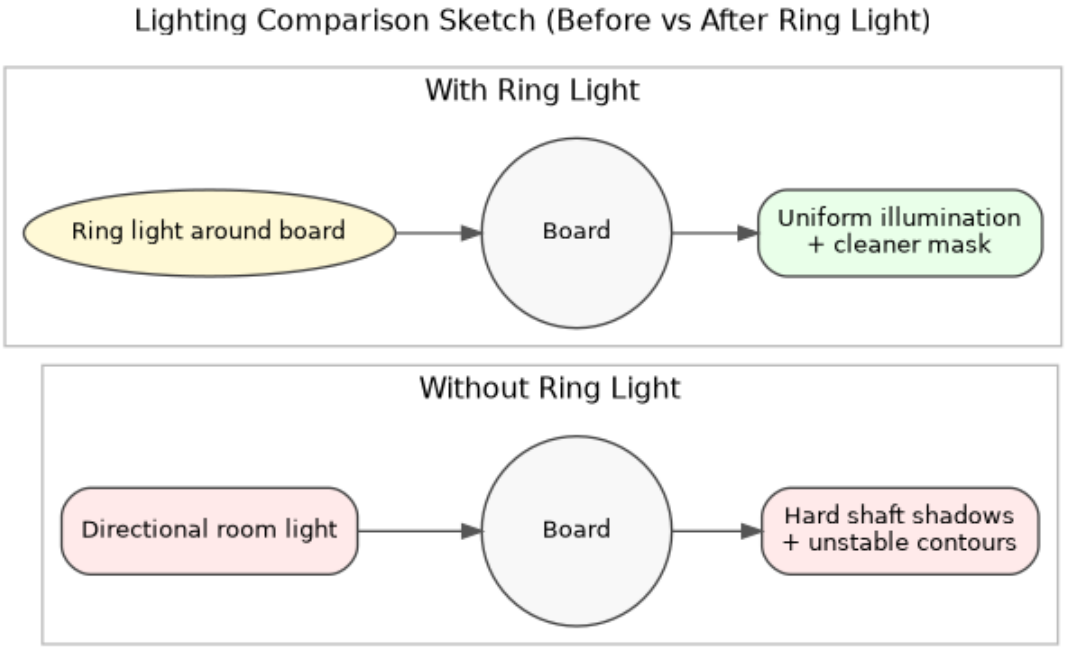


Figure 11.: Conceptual before/after sketch of lighting impact on shadow behavior and contour stability.

6.6. Experimental Results

6.6.1. Successful Detection Scenarios

The strongest results were observed in the following scenarios:

- Good calibration with stable camera geometry.
- Uniform ring-light illumination with minimal cast shadows.
- Camera placement that keeps the full board in frame while maximizing board scale.
- Angle near the empirically chosen range ($\approx 45°$ to $50°$), balancing contour clarity and sector readability.

Under these conditions, the detector produced stable contour masks and consistent sector assignment across repeated throws.

6.6.2. Failure Scenarios

Observed failures were repeatable and can be grouped as follows:

- **Calibration error:** Incorrect ring/sector alignment shifts classifications into neighboring fields.
- **Boundary ambiguity:** Large or irregular contours exactly on sector borders can flip between adjacent sectors.
- **Bounce-out behavior:** Non-persistent impacts are not represented in the final static frame and are therefore missed.
- **Mount vibration:** If the camera is mechanically coupled to board vibrations, global frame motion causes detector confusion and can suppress valid detections.

6.6.3. Reproducibility Checklist

To improve repeatability of future experiments, the following protocol was derived from the final evaluation setup.

Listing 17: Minimal reproducibility protocol for future test repetitions

```
Verify camera mount rigidity and fixed board alignment.
Confirm ring-light operation and remove directional room light hotspots.
Re-run calibration and save configuration before test session.
Execute mixed throw set: center, near-edge, out-of-bounds, rapid succession.
Log each mismatch under one of the defined failure categories.
Re-test after any parameter adjustment to detect regressions.
```

Table 17.: Failure taxonomy used for structured error logging.

Category	Operational Description
Calibration error	Sector mapping offset due to inaccurate board reference alignment
Boundary ambiguity	Neighbor-sector flip for impacts exactly on ring/sector borders
Bounce-out limitation	No persistent contour remains after non-sticking impact
Vibration-induced instability	Mount/board coupling causes global frame motion and detector overload

7. Summary

7.1. Review of Objectives

The overall aim of this thesis has been to integrate a low-cost system for automatic scoring within a game of steel-tip darts, eliminating manual score tracking but retaining the "feeling" of a traditional game. This has been successfully accomplished through a completed prototype.

While there is a high degree of coupling within this project, there is a distinct separation of modules in the end. This has been useful for both maintenance and performance.

7.1.1. Functional Objectives

Web Interface and Real-Time Interaction

One of the key objectives of this project has been to design a system such that it can be interacted with through a variety of devices within a local environment, yet present a single source of truth for users. This has been accomplished through a completed frontend, where a variety of route-based workflows, reactive stores, and websockets have been implemented to present a seamless experience to users.

While interacting with this system, it has also been important to consider how users will interact with it under suboptimal conditions. This has been accomplished through a variety of states within a UI, ensuring operator confidence under a variety of scenarios.

This has been an important part of ensuring that real-time interaction is not simply a transport-layer concern but also a concern for states and interfaces.

Game Logic and State Management

Support for 301/501 gameplay rules, bust handling, and checkout constraint checking was provided through the frontend ruleset engine. The choice of using deterministic state transitions for modeling the game resulted in a much simpler and more deterministic

set of test cases. In addition, there was less ambiguity in edge cases since everything is driven through ordered throw events.

From a software engineering point of view, this part of the system illustrates how simple reactive state transitions are adequate for modeling simple reactive systems where transitions are explicit. In this project, there was no need for using complex orchestration tools for creating a robust state machine.

Statistics and Persistence-Oriented Features

Another goal for this thesis was to extend the system beyond simple scoring and into more useful retrospective analysis. As a result, the final implementation includes a persistent match history model and analytics views with useful metrics.

The chosen split computation strategy for this problem was found to be useful. Relational aggregations are performed backend-side, and lightweight derivative metrics are performed frontend-side. This results in compact network traffic and allows for immediate update of the UI when switching between players or revisiting previous sessions. In addition, defensive programming for handling sparse or missing data points was useful in preventing undefined results in the analytics view.

7.2. Technical Evaluation

7.2.1. System-Level Observations

The evaluation chapter illustrates how subsystem quality is highly context-dependent. In particular, the quality of the detection results is highly dependent on the quality of the physical setup, whereas the quality of the frontend is highly dependent on clear semantics and correction workflows. The overall quality of the finished system is best achieved under circumstances where both factors are met.

A practical consequence of this is that users experience end-to-end quality through continuity of interaction, not through the raw results of the algorithms. This was a particularly relevant concept for the recreational deployment goals, where trust of the overall system is more important than individual benchmark results.

7.2.2. Strengths of the Final Architecture

The final architecture offers four strengths:

- **Low Operational Barrier**: The solution only requires consumer-grade hardware and local browser clients.
- **Fast Interaction Loop**: The solution is event-driven, allowing for rapid gameplay response.
- **Recoverability**: The solution includes correction workflows and error states, limiting the impact of detection errors.
- **Analytical Value**: The solution allows for the interpretation of gameplay statistics beyond the simple result.

7.2.3. Remaining Constraints

While the solution has been successful, there are several constraints. Vision reliability can be compromised by extreme geometric or lighting irregularities. The solution is also optimized for local use, not for a more robust distribution. In terms of application, the solution's analytical depth is limited, although this is a conscious choice for simplicity and response. These constraints, although present, are well beyond the scope of this project.

7.3. Reflection on Division of Work

The appendix established a collaboration model where web interface, vision, statistics, and persistence are primarily the first author's responsibility, and camera/hardware, as well as detection, are primarily the second author's responsibility. The structure of this document now more clearly reflects this model, where discussion of the frontend, integration, and statistics is primarily the first author's responsibility, and discussion of the camera, hardware, and detection is primarily the second author's responsibility.

This alignment has helped to improve the technical coherence and traceability of the contributions. It has also reduced the ambiguity in the mixed sections, where software integration topics like event contracts, synchronization, and correction workflows are now included in the first author domain, while physically constrained detection behavior remains in the second author domain.

Focus Area	Implemented Outcome
Web interface and interaction flow	Multi-route browser UI for setup, game, and analytics with reactive state continuity
Real-time event presentation	WebSocket-driven score updates with explicit connection/error feedback
Game rules and state transitions	Deterministic handling of 301/501 logic, bust rollback, checkout checks, and undo
Statistics and persistence integration	Persistent match history with interpretable player-centric metrics and robust empty-data handling

Table 18.: Contribution-oriented perspective on implemented software outcomes.

7.4. Potential Enhancements

Richer Analytics and Session Intelligence

Further improvements to the software could be made to the statistics module, for example, by providing rolling averages, distribution of checkouts in the checkout class, and consistency trends in specific segments. A particularly interesting potential is in the progression analysis between sessions, in which players can compare their recent performance to their own baseline.

Operator-Centric Configuration UX

Tuning and calibration of the game and the environment could be significantly simplified through a unified operator interface for live confidence, thresholds, and environment diagnostics. Instead of just providing end-state results, such an interface would allow for early detection of problematic states and reduce the calibration time for operators without game expertise.

Enhanced Event Integrity

For more general application scenarios, event integrity can also be provided using sequence identifiers, action logs, and reconciliation processes after reconnecting. Such

features are not necessarily required for local usage in a recreational context but may become important in noisier network environments or in multi-session usage scenarios where sessions may be long-running.

Machine Learning as a Complementary Path

Data-driven approaches to detection should also remain a viable future path, particularly for challenging scenarios such as heavy occlusion and intricate reflections. Nevertheless, such a path should be pursued as a complement to rather than a replacement for the current deterministic approach.

7.5. Final Conclusion

The present work has shown through a concrete implementation that a system for automatic dart scoring can indeed be constructed as a viable software/hardware solution, even without relying on any special sensors. The implemented system indeed works as intended by combining live hits, a functional web interface, deterministic gameplay, and persistent analytics.

From a personal perspective, perhaps the biggest contribution of this work is to show that frontend architecture indeed plays a role in overall system quality. Real-time rendering, explicit states, and correction paths are not just niceties but rather functional aspects of a system that can indeed make data-driven approaches to detection trustworthy. With this foundation now in place, further work can indeed improve aspects such as robustness and analytical depth without compromising on aspects such as simplicity, responsiveness, and accessibility.

Bibliography

[1] M. Osadchy and D. Keren, "Incorporating the Boltzmann prior in object detection using SVM," *Proceedings of the IEEE Computer Society Conference on Computer Vision and Pattern Recognition*, vol. 2, pp. 2095–2101, 2006.

[2] G. Welch and G. Bishop, "An Introduction to the Kalman Filter," University of North Carolina at Chapel Hill, Tech. Rep. TR 95-041, 2006. [Online]. Available: http://www.cs.unc.edu/$\sim$welch/media/pdf/kalman_intro.pdf

[3] R. Hartley and A. Zisserman, "Multiple View Geometry in Computer Vision (Cited by: 11343)," *Cambridge University Press*, vol. 2, no. 2, p. 672, 2004. [Online]. Available: http://www.robots.ox.ac.uk/$\sim$vgg/hzbook/

[4] C. W. Schwingshackl, L. Massei, C. Zang, and D. J. Ewins, "A constant scanning LDV technique for cylindrical structures: Simulation and measurement," *Mechanical Systems and Signal Processing*, vol. 24, no. 2, pp. 394–405, feb 2010.

[5] Z. Zivkovic, "Improved adaptive Gaussian mixture model for background subtraction," *Proceedings - International Conference on Pattern Recognition*, vol. 2, pp. 28–31, 2004.

[6] J. Canny, "A Computational Approach to Edge Detection," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. PAMI-8, no. 6, pp. 679–698, 1986.

[7] R. O. Duda and P. E. Hart, "Use of the Hough transformation to detect lines and curves in pictures," *Communications of the ACM*, vol. 15, no. 1, pp. 11–15, jan 1972. [Online]. Available: https://dl.acm.org/doi/10.1145/361237.361242

[8] Z. Zhang, "A flexible new technique for camera calibration," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 22, no. 11, pp. 1330–1334, nov 2000.

[9] R. Szeliski, *Computer Vision*, ser. Texts in Computer Science. Cham: Springer International Publishing, 2022. [Online]. Available: https://link.springer.com/10.1007/978-3-030-34372-9

[10] C. R. Harris *et al.*, "Array programming with NumPy," *Nature 2020 585:7825*, vol. 585, no. 7825, pp. 357–362, sep 2020. [Online]. Available: https://www.nature.com/articles/s41586-020-2649-2

[11] P. Viola and M. Jones, "Rapid Object Detection using a Boosted Cascade of Simple Features," in *Proceedings of the 2001 IEEE Computer Society Conference on Computer Vision and Pattern Recognition.* IEEE, 2001, pp. I–511–I–518.

[12] N. Otsu, "THRESHOLD SELECTION METHOD FROM GRAY-LEVEL HISTOGRAMS." *IEEE Trans Syst Man Cybern*, vol. SMC-9, no. 1, pp. 62–66, 1979.

[13] S. Garrido-Jurado, R. Muñoz-Salinas, F. J. Madrid-Cuevas, and M. J. Marín-Jiménez, "Automatic generation and detection of

highly reliable fiducial markers under occlusion," *Pattern Recognition*, vol. 47, no. 6, pp. 2280–2292, jun 2014. [Online]. Available: https://www.researchgate.net/publication/260251570_Automatic_generation_and_detection_of_highly_reliable_fiducial_markers_under_occlusion

[14] C. Stauffer and W. E. Grimson, "Adaptive background mixture models for real-time tracking," *Proceedings of the IEEE Computer Society Conference on Computer Vision and Pattern Recognition*, vol. 2, pp. 246–252, 1999.

List of Figures

List of Tables

List of Listings

Appendix

A. Project Management

A.1. Project Timeline

A.1.1. Milestones

Development Phases

The project is divided into various primary phases, each of which ends at a predetermined milestone. This ensures that the project follows a well-structured pattern from the foundational research to the final working prototype.

- **Design Prototype Completed (09.07.2025):** This milestone marks the completion of the web interface design. The design prototype includes wireframes and mockups for all the significant screens, such as the scoreboard, statistics, and the game itself. It is validated through user feedback sessions to ensure the project meets the criteria for usability standards.
- **Image Capture Prototype Created (11.07.2025):** A functional prototype for the vision subsystem is developed, including the configuration of the camera with the chosen hardware, the capture of images from the chosen point of view, and the implementation of the initial image acquisition routines to ensure the quality of the images obtained, including the lighting conditions.
- **Initial Game Logic Implementation Completed (16.07.2025):** This milestone marks the implementation of the rules for the primary game modes, such as 501 or Double Out. It ensures the correct functionality of the score, the turns, and the winning conditions without the vision subsystem.
- **Game Persistence Implemented (21.07.2025):** This milestone marks the implementation of the storage functionality, including the database or file-based storage, which will allow the project to store the game history, the profiles of the

players, and the match results, laying the foundation for the implementation of the statistical analysis features.

- **Throw Detection Completed (01.08.2025):** This is one of the more important technical milestones, where the integration of the vision subsystem with the game logic is successfully implemented. It detects the placement of the new dart on the board and identifies the score segment with sufficient accuracy.
- **Statistical Evaluations Completed (04.08.2025):** The final feature milestone. The system is now able to generate and display meaningful statistics derived from stored game data, e.g., per-dart averages, checkout percentages, and score heatmaps, thus completing the full feature set of the proposed prototype.

A.2. Task Distribution

A.2.1. Individual Contributions

The present thesis is a collaborative effort that reflects a well-defined division of labor that takes full advantage of the different skill sets of the collaborators. The work is divided into two main domains of expertise: the vision and hardware system, and the web application and user experience.

Luca Zinhobel: Vision and Hardware System

Luca is in charge of the entire backend of the physical detection system. The tasks assigned to this collaborator are dedicated to overcoming the major problems of computer vision and ensuring hardware adequacy for this purpose. The specific contributions of this collaborator are:

- **Hardware Selection and Setup:** Research and select an optimal camera model and establish hardware setup and camera perspective for capturing images appropriately.
- **Vision Algorithm Development:** Research and implement required image processing techniques for detecting darts and determining their scores.
- **System Calibration:** Design and implement manual and marker-based calibration methods for accurately mapping images onto the dartboard.

- **Integration:** Ensure that results from the vision module are provided in a format that is easily accessible by the application logic.

Marvin Anderson: Web Application and User Experience

Marvin is in charge of the entire user interface of the system. The tasks assigned to this collaborator are dedicated to developing an intuitive, informative, and responsive user interface that will act as a control panel for the game and display the score. The specific contributions of this collaborator are:

- **Web Interface Development:** The development of a user-friendly and responsive web-based interface that may be accessed through various devices such as smartphones, tablets, and computers.
- **Real-Time Display:** Integration of appropriate technologies to update the web interface with the latest information from the detection system in real-time so that the state is reflected simultaneously across all connected devices.
- **Game Logic and State Management:** Development of the core logic for different types of games (301, 501, Double Out) and the overall state management for the application (players, turns, etc.).
- **Statistics and Persistence:** Development of the database schema to store the data appropriately and the frontend components to display meaningful statistics for the players.